

第 2 章 資料清理與轉換(tidyverse)

Table of Contents

大綱:	4
2.1 tidy data 概念與常見資料問題(缺值、重複、型態)	4
Tidy Data 簡介	4
Tidy Data 範例	4
不符合 Tidy Data 範例	5
資料品質(Data Quality)	6
1.遺漏值(Missing Values)	6
填補遺漏值.....	28
方法一：平均數.....	28
方法二：中位數.....	28
方法三：固定值.....	29
2.重複值(Duplicate Records).....	29
找出重複資料列.....	29
查看重複資料	29
刪除重複資料列.....	29
刪除重複資料列 {dplyr}	30
3.資料型態錯誤(Incorrect Data Types)	30
常見資料型態錯誤	30
字串轉換為日期.....	31
字串轉換為因子 (Factor)	31
因子轉換字串	31
2.2 dplyr : filter/select/mutate/summarise/group_by	31

管線運算子(%>%).....	31
認識 dplyr 套件.....	32
flights 2013 年 NYC 機場航班資料, 33 萬筆資料.....	32
dplyr 套件重要函數.....	38
filter 條件式篩選資料.....	38
slice 列的指標篩選資料.....	40
arrange 排序.....	42
select 選取行.....	43
select 選取行, 使用冒號(:)表示連續範圍.....	44
select 選取行, 使用負號(-)表示刪除.....	44
select 選取行並且更改欄位名稱.....	45
select 選取行, 使用 starts_with 等參數.....	45
rename 更改欄位名稱.....	48
distinct 選取不重覆資料.....	49
mutate 新增欄位.....	51
summarise - 計算加總.....	54
summarise - 平均起飛延誤.....	54
summarise - 多個統計量.....	54
group_by + summarise.....	54
group_by 多群組 + summarise.....	55
計算航空公司的平均延誤時間, 由大至小排序.....	56
dplyr in R Cheat Sheet.....	57
熟悉開放資料集於 dplyr 應用.....	57
✿ 2.3 tidyr : pivot_longer/pivot_wider、分欄/合欄.....	57
長寬資料簡介.....	57

tidyr + ggplot2 -> 使用 tidyverse 整合套件	59
近十年國防預算結構分析	62
長資料(1 行軍事投資,作業維持,人員維持,其他)轉換為寬資料(4 行)	66
方法 1 不使用管線操作	66
方法 2 使用 %>%	66
國防預算結構統計圖	67
❀ 2.4 合併資料：join 家族與鍵值設計	68
合併簡介	68
範例 1 left_join - 左表新增右表的 1 個欄位	69
範例 2 left_join - 左表新增右表的多個欄位	70
範例 3 left_join - 右表的欄位找不到	70
❀ 2.5 練習：完成一份可分析的乾淨資料集	71
參考 Open Data 等資料，完成 Tidy Data。	71
熟悉以下資料集之資料操作	71
1. AirPassengers 空中旅客人數資料集 ——	71
2. Titanic 1912 年鐵達尼號資料集	73
3. airquality 1973 年紐約空氣品質資料集	80
4. iris 鳶尾花資料集——	85
5. mtcars 汽車資料集	90
6. penguins 南極洲帕爾默站企鵝	94

```
# file      : r_data_science_02_tidyverse.Rmd
# title     : R 入門實作班
# subtitle  : 2. 資料清理與轉換(tidyverse)
# author    : Ming-Chang Lee
# date      : 2026.08.09
# YouTube   : https://www.youtube.com/@alan9956
# RWEPA     : http://rwepa.blogspot.tw/
# GitHub    : https://github.com/rwepa
# Email     : alan9956@gmail.com
```

大綱:

- 2.1 tidy data 概念與常見資料問題(缺值、重複、型態)
- 2.2 dplyr : filter/select/mutate/summarise/group_by
- 2.3 tidyr : pivot_longer/pivot_wider、分欄/合欄
- 2.4 合併資料 : join 家族與鍵值設計
- 2.5 練習 : 完成一份可分析的乾淨資料集

✿ 2.1 tidy data 概念與常見資料問題(缺值、重複、型態)

Tidy Data 簡介

Tidy Data(整潔資料)是由資料科學 Hadley Wickham 提出的資料整理原則，也是 tidyverse 生態系的核心概念。

符合 Tidy Data 的資料，具有三項基本原則：

1. 每個變數(Variable)使用一個欄(Column)。
2. 每筆觀測(Observation)使用一行(Row)。
3. 每個資料值(Value)使用一個儲存格(Cell)。

Tidy Data 範例

```
library(tibble)

student <- tibble(
  ID = c(1, 2, 3, 4),
  Name = c("ALAN", "BOB", "Cathy", "David"),
  Gender = c("F", "M", "F", "M"),
  Math = c(95, 70, 80, 90),
  English = c(80, 85, 90, 77)
)
```

```
print(student)

## # A tibble: 4 × 5
##       ID Name  Gender  Math English
##   <dbl> <chr> <chr>   <dbl>   <dbl>
## 1     1  ALAN   F       95      80
## 2     2   BOB   M       70      85
## 3     3  Cathy F       80      90
## 4     4 David M       90      77
```

此資料廣義而言符合 Tidy Data 原則：

- ID：變數。
- Name：變數。
- Gender：變數。
- Math：變數。
- English：變數。
- 每一列代表一位學生。
- 每個儲存格只有一個值。

不符合 Tidy Data 範例

上述範例嚴格來說，不完全符合 Tidy Data 的概念。

1. 每一欄代表一個變數（Variable），不符合 Tidy 原則，Math、English 其實都是同一個變數「科目」的不同值。
2. 每一列代表一筆觀測值（Observation），不符合 Tidy 原則，每位學生有兩筆成績，但目前只佔一列。
3. 每一個儲存格只有一個值（Value），完全符合，每個儲存格只有一個資料值。

上例中的 Math, English 修改為 2 個欄位 (Subject, Score)，修改後為長資料(Long data)，修改後結果如下表所示。

```
# > student_long
## # A tibble: 8 × 5
##       ID Name  Gender Subject Score
##   <dbl> <chr> <chr>   <chr>   <dbl>
## 1     1  ALAN   F      Math     95
## 2     1  ALAN   F      English  80
## 3     2   BOB   M      Math     70
## 4     2   BOB   M      English  85
## 5     3  Cathy F      Math     80
## 6     3  Cathy F      English  90
```

# 7	4 David M	Math	90
# 8	4 David M	English	77

資料品質(Data Quality)

實務上，資料分析約 60%~80% 的時間都花在資料前處理(Data Cleaning)，常見問題包括：

1.遺漏值(Missing Values)。2.重複值(Duplicate Records)。3.資料型態錯誤(Incorrect Data Types)。

1.遺漏值(Missing Values)

- 遺漏值在 R 中顯示為 NA, 不是空白.

```
data(Cars93, package = "MASS")
df <- Cars93
print(head(df))
```

```
##   Manufacturer  Model   Type Min.Price Price Max.Price MPG.city MP
G.highway
## 1      Acura Integra  Small    12.9  15.9    18.8    25
31
## 2      Acura  Legend Midsize    29.2  33.9    38.7    18
25
## 3      Audi    90 Compact    25.9  29.1    32.3    20
26
## 4      Audi   100 Midsize    30.8  37.7    44.6    19
26
## 5      BMW    535i Midsize    23.7  30.0    36.2    22
30
## 6     Buick Century Midsize    14.2  15.7    17.3    22
31
##           AirBags DriveTrain Cylinders EngineSize Horsepower  RPM
## 1             None      Front         4         1.8      140 6300
## 2 Driver & Passenger      Front         6         3.2      200 5500
## 3      Driver only      Front         6         2.8      172 5500
## 4 Driver & Passenger      Front         6         2.8      172 5500
## 5      Driver only      Rear         4         3.5      208 5700
## 6      Driver only      Front         4         2.2      110 5200
##   Rev.per.mile Man.trans.avail Fuel.tank.capacity Passengers Length
Wheelbase
## 1      2890             Yes             13.2          5      177
102
## 2      2335             Yes             18.0          5      195
115
## 3      2280             Yes             16.9          5      180
102
## 4      2535             Yes             21.1          6      193
106
```

```

## 5      2545      Yes      21.1      4      186
      109
## 6      2565      No      16.4      6      189
      105
##   Width Turn.circle Rear.seat.room Luggage.room Weight  Origin
      Make
## 1      68      37      26.5      11      2705 non-USA Acura
      Integra
## 2      71      38      30.0      15      3560 non-USA Acur
a Legend
## 3      67      37      28.0      14      3375 non-USA
      Audi 90
## 4      70      37      31.0      17      3405 non-USA
Audi 100
## 5      69      39      27.0      13      3640 non-USA
BMW 535i
## 6      69      41      28.0      16      2880      USA Buick
      Century

# 資料類別: data.frame 資料框物件
class(df)

## [1] "data.frame"

# 資料結構, 包括Factor 因子(資料型態是字元 character), num 數值, int 整數.
str(df)

## 'data.frame':   93 obs. of  27 variables:
## $ Manufacturer   : Factor w/ 32 levels "Acura","Audi",...: 1 1 2
2 3 4 4 4 4 5 ...
## $ Model          : Factor w/ 93 levels "100","190E","240",...: 49
56 9 1 6 24 54 74 73 35 ...
## $ Type           : Factor w/ 6 levels "Compact","Large",...: 4 3
1 3 3 3 2 2 3 2 ...
## $ Min.Price      : num  12.9 29.2 25.9 30.8 23.7 14.2 19.9 22.6
26.3 33 ...
## $ Price          : num  15.9 33.9 29.1 37.7 30 15.7 20.8 23.7 26.
3 34.7 ...
## $ Max.Price      : num  18.8 38.7 32.3 44.6 36.2 17.3 21.7 24.9
26.3 36.3 ...
## $ MPG.city       : int   25 18 20 19 22 22 19 16 19 16 ...
## $ MPG.highway    : int   31 25 26 26 30 31 28 25 27 25 ...
## $ AirBags        : Factor w/ 3 levels "Driver & Passenger",...: 3
1 2 1 2 2 2 2 2 2 ...
## $ DriveTrain     : Factor w/ 3 levels "4WD","Front",...: 2 2 2 2
3 2 2 3 2 2 ...
## $ Cylinders       : Factor w/ 6 levels "3","4","5","6",...: 2 4 4
4 2 2 4 4 4 5 ...
## $ EngineSize     : num   1.8 3.2 2.8 2.8 3.5 2.2 3.8 5.7 3.8 4.9
...

```

```
## $ Horsepower      : int  140 200 172 172 208 110 170 180 170 200
...
## $ RPM             : int  6300 5500 5500 5500 5700 5200 4800 4000
4800 4100 ...
## $ Rev.per.mile    : int  2890 2335 2280 2535 2545 2565 1570 1320
1690 1510 ...
## $ Man.trans.avail : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 1 1
1 1 1 ...
## $ Fuel.tank.capacity: num  13.2 18 16.9 21.1 21.1 16.4 18 23 18.8 1
8 ...
## $ Passengers      : int   5 5 5 6 4 6 6 6 5 6 ...
## $ Length          : int  177 195 180 193 186 189 200 216 198 206
...
## $ Wheelbase       : int  102 115 102 106 109 105 111 116 108 114
...
## $ Width           : int   68 71 67 70 69 69 74 78 73 73 ...
## $ Turn.circle     : int   37 38 37 37 39 41 42 45 41 43 ...
## $ Rear.seat.room  : num   26.5 30 28 31 27 28 30.5 30.5 26.5 35 ...
## $ Luggage.room    : int   11 15 14 17 13 16 17 21 14 18 ...
## $ Weight          : int  2705 3560 3375 3405 3640 2880 3470 4105
3495 3620 ...
## $ Origin          : Factor w/ 2 levels "USA","non-USA": 2 2 2 2 2
1 1 1 1 1 ...
## $ Make            : Factor w/ 93 levels "Acura Integra",...: 1 2 4
3 5 6 7 9 8 10 ...
```

Rear.seat.room 等欄位有NA

summary(df)

```
##      Manufacturer      Model      Type      Min.Price      Price
## Chevrolet: 8      100      : 1      Compact:16      Min.      : 6.70      Min.      :
7.40
## Ford      : 8      190E      : 1      Large      :11      1st Qu.:10.80      1st Qu.:1
2.20
## Dodge      : 6      240      : 1      Midsize:22      Median :14.70      Median :1
7.70
## Mazda      : 5      300E      : 1      Small      :21      Mean      :17.13      Mean      :1
9.51
## Pontiac    : 5      323      : 1      Sporty      :14      3rd Qu.:20.30      3rd Qu.:2
3.30
## Buick      : 4      535i      : 1      Van          : 9      Max.      :45.40      Max.      :6
1.90
## (Other)    :57      (Other):87

##      Max.Price      MPG.city      MPG.highway      AirBag
s
## Min.      : 7.9      Min.      :15.00      Min.      :20.00      Driver & Passenger:1
6
## 1st Qu.:14.7      1st Qu.:18.00      1st Qu.:26.00      Driver only      :4
```



```

3
## Median :19.6   Median :21.00   Median :28.00   None           :3
4
## Mean    :21.9   Mean    :22.37   Mean    :29.09

## 3rd Qu.:25.3   3rd Qu.:25.00   3rd Qu.:31.00

## Max.     :80.0   Max.     :46.00   Max.     :50.00

##

## DriveTrain  Cylinders      EngineSize      Horsepower      RPM
## 4WD  :10   3      : 3   Min.    :1.000   Min.    : 55.0   Min.    :3800
## Front:67   4      :49   1st Qu.:1.800   1st Qu.:103.0   1st Qu.:4800
## Rear :16   5      : 2   Median  :2.400   Median  :140.0   Median  :5200
##                               6      :31   Mean    :2.668   Mean    :143.8   Mean    :5281
##                               8      : 7   3rd Qu.:3.300   3rd Qu.:170.0   3rd Qu.:5750
##                               rotary: 1   Max.    :5.700   Max.    :300.0   Max.    :6500
##

## Rev.per.mile  Man.trans.avail  Fuel.tank.capacity  Passengers
## Min.    :1320   No :32              Min.    : 9.20      Min.    :2.000
## 1st Qu.:1985   Yes:61             1st Qu.:14.50      1st Qu.:4.000
## Median  :2340              Median :16.40      Median  :5.000
## Mean    :2332              Mean   :16.66      Mean    :5.086
## 3rd Qu.:2565              3rd Qu.:18.80      3rd Qu.:6.000
## Max.    :3755              Max.    :27.00      Max.    :8.000
##

## Length      Wheelbase      Width      Turn.circle
## Min.    :141.0   Min.    : 90.0   Min.    :60.00   Min.    :32.00
## 1st Qu.:174.0   1st Qu.: 98.0   1st Qu.:67.00   1st Qu.:37.00
## Median  :183.0   Median :103.0   Median :69.00   Median :39.00
## Mean    :183.2   Mean    :103.9   Mean    :69.38   Mean    :38.96
## 3rd Qu.:192.0   3rd Qu.:110.0   3rd Qu.:72.00   3rd Qu.:41.00
## Max.    :219.0   Max.    :119.0   Max.    :78.00   Max.    :45.00
##

## Rear.seat.room  Luggage.room      Weight      Origin
## Make
## Min.    :19.00   Min.    : 6.00   Min.    :1695   USA    :48   Acura I
ntegra: 1
## 1st Qu.:26.00   1st Qu.:12.00   1st Qu.:2620   non-USA:45   Acura L

```

```

legend : 1
## Median :27.50 Median :14.00 Median :3040 Audi 10
0 : 1
## Mean :27.83 Mean :13.89 Mean :3073 Audi 90
: 1
## 3rd Qu.:30.00 3rd Qu.:15.00 3rd Qu.:3525 BMW 535
i : 1
## Max. :36.00 Max. :22.00 Max. :4105 Buick C
entury: 1
## NAs :2 NAs :11 (Other)
:87

# 查看遺漏值
is.na(df[1:3, 1:5])

## Manufacturer Model Type Min.Price Price
## 1 FALSE FALSE FALSE FALSE FALSE
## 2 FALSE FALSE FALSE FALSE FALSE
## 3 FALSE FALSE FALSE FALSE FALSE

# 每欄遺漏值數量, Rear.seat.room, Luggage.room 有遺漏值
colSums(is.na(df))

## Manufacturer Model Type Mi
n.Price
## 0 0 0
0
## Price Max.Price MPG.city MPG.
highway
## 0 0 0
0
## AirBags DriveTrain Cylinders Eng
ineSize
## 0 0 0
0
## Horsepower RPM Rev.per.mile Man.tran
s.avail
## 0 0 0
0
## Fuel.tank.capacity Passengers Length Wh
eelbase
## 0 0 0
0
## Width Turn.circle Rear.seat.room Lugga
ge.room
## 0 0 2
11
## Weight Origin Make
## 0 0 0

```

每列遺漏值數量

```
rowSums(is.na(df))
```

```
## 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26
## 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 2 0 0 0 0
0 0 1
## 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
50 51 52
## 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0
## 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
76 77 78
## 0 0 0 1 2 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0
0 0 0
## 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93
## 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0
```

找出含 NA 的資料列

```
df[!complete.cases(df), ]
```

```
##      Manufacturer      Model      Type Min.Price Price Max.Price MPG.city
## 16      Chevrolet Lumina_APV      Van      14.7  16.3      18.0      18
## 17      Chevrolet      Astro      Van      14.7  16.6      18.6      15
## 19      Chevrolet      Corvette Sporty      34.6  38.0      41.5      17
## 26          Dodge      Caravan      Van      13.6  19.0      24.4      17
## 36          Ford      Aerostar      Van      14.5  19.9      25.3      15
## 56          Mazda      MPV      Van      16.6  19.1      21.7      18
## 57          Mazda      RX-7 Sporty      32.5  32.5      32.5      17
## 66          Nissan      Quest      Van      16.7  19.1      21.5      17
## 70      Oldsmobile Silhouette      Van      19.5  19.5      19.5      18
## 87          Toyota      Previa      Van      18.9  22.7      26.6      18
## 89      Volkswagen      Eurovan      Van      16.6  19.7      22.7      17
##      MPG.highway      AirBags DriveTrain Cylinders EngineSize Horsepower
r RPM
## 16          23          None      Front          6          3.8          17
0 4800
## 17          20          None      4WD          6          4.3          16
5 4000
## 19          25 Driver only      Rear          8          5.7          30
0 5000
## 26          21 Driver only      4WD          6          3.0          14
2 5000
## 36          20 Driver only      4WD          6          3.0          14
5 4800
## 56          24          None      4WD          6          3.0          15
5 5000
## 57          25 Driver only      Rear      rotary          1.3          25
5 6500
## 66          23          None      Front          6          3.0          15
```

1	4800						
##	70	23	None	Front	6	3.8	17
0	4800						
##	87	22	Driver only	4WD	4	2.4	13
8	5000						
##	89	21	None	Front	5	2.5	10
9	4500						
##		Rev.per.mile	Man.trans.avail	Fuel.tank.capacity	Passengers	Length	
		Wheelbase					
##	16	1690	No	20.0	7	178	
	110						
##	17	1790	No	27.0	8	194	
	111						
##	19	1450	Yes	20.0	2	179	
	96						
##	26	1970	No	20.0	7	175	
	112						
##	36	2080	Yes	21.0	7	176	
	119						
##	56	2240	No	19.6	7	190	
	110						
##	57	2325	Yes	20.0	2	169	
	96						
##	66	2065	No	20.0	7	190	
	112						
##	70	1690	No	20.0	7	194	
	110						
##	87	2515	Yes	19.8	7	187	
	113						
##	89	2915	Yes	21.1	7	187	
	115						
##		Width	Turn.circle	Rear.seat.room	Luggage.room	Weight	Origin
##	16	74	44	30.5	NA	3715	USA
##	17	78	42	33.5	NA	4025	USA
##	19	74	43	NA	NA	3380	USA
##	26	72	42	26.5	NA	3705	USA
##	36	72	45	30.0	NA	3735	USA
##	56	72	39	27.5	NA	3735	non-USA
##	57	69	37	NA	NA	2895	non-USA
##	66	74	41	27.0	NA	4100	non-USA
##	70	74	44	30.5	NA	3715	USA
##	87	71	41	35.0	NA	3785	non-USA
##	89	72	38	34.0	NA	3960	non-USA
##			Make				
##	16		Chevrolet Lumina_APV				
##	17		Chevrolet Astro				
##	19		Chevrolet Corvette				
##	26		Dodge Caravan				
##	36		Ford Aerostar				
##	56		Mazda MPV				

```
## 57 Mazda RX-7
## 66 Nissan Quest
## 70 Oldsmobile Silhouette
## 87 Toyota Previa
## 89 Volkswagen Eurovan
```

刪除遺漏值, *df* 資料沒有改變, 只是顯示執行結果

如果希望

`na.omit(df)`

##	Manufacturer	Model	Type	Min.Price	Price	Max.Price	MP
G.city							
## 1	Acura	Integra	Small	12.9	15.9	18.8	
25							
## 2	Acura	Legend	Midsize	29.2	33.9	38.7	
18							
## 3	Audi	90	Compact	25.9	29.1	32.3	
20							
## 4	Audi	100	Midsize	30.8	37.7	44.6	
19							
## 5	BMW	535i	Midsize	23.7	30.0	36.2	
22							
## 6	Buick	Century	Midsize	14.2	15.7	17.3	
22							
## 7	Buick	LeSabre	Large	19.9	20.8	21.7	
19							
## 8	Buick	Roadmaster	Large	22.6	23.7	24.9	
16							
## 9	Buick	Riviera	Midsize	26.3	26.3	26.3	
19							
## 10	Cadillac	DeVille	Large	33.0	34.7	36.3	
16							
## 11	Cadillac	Seville	Midsize	37.5	40.1	42.7	
16							
## 12	Chevrolet	Cavalier	Compact	8.5	13.4	18.3	
25							
## 13	Chevrolet	Corsica	Compact	11.4	11.4	11.4	
25							
## 14	Chevrolet	Camaro	Sporty	13.4	15.1	16.8	
19							
## 15	Chevrolet	Lumina	Midsize	13.4	15.9	18.4	
21							
## 18	Chevrolet	Caprice	Large	18.0	18.8	19.6	
17							
## 20	Chrylser	Concorde	Large	18.4	18.4	18.4	
20							
## 21	Chrysler	LeBaron	Compact	14.5	15.8	17.1	
23							
## 22	Chrysler	Imperial	Large	29.5	29.5	29.5	
20							

## 23 29	Dodge	Colt	Small	7.9	9.2	10.6
## 24 23	Dodge	Shadow	Small	8.4	11.3	14.2
## 25 22	Dodge	Spirit	Compact	11.9	13.3	14.7
## 27 21	Dodge	Dynasty	Midsize	14.8	15.6	16.4
## 28 18	Dodge	Stealth	Sporty	18.5	25.8	33.1
## 29 29	Eagle	Summit	Small	7.9	12.2	16.5
## 30 20	Eagle	Vision	Large	17.5	19.3	21.2
## 31 31	Ford	Festiva	Small	6.9	7.4	7.9
## 32 23	Ford	Escort	Small	8.4	10.1	11.9
## 33 22	Ford	Tempo	Compact	10.4	11.3	12.2
## 34 22	Ford	Mustang	Sporty	10.8	15.9	21.0
## 35 24	Ford	Probe	Sporty	12.8	14.0	15.2
## 37 21	Ford	Taurus	Midsize	15.6	20.2	24.8
## 38 18	Ford Crown_Victoria		Large	20.1	20.9	21.7
## 39 46	Geo	Metro	Small	6.7	8.4	10.0
## 40 30	Geo	Storm	Sporty	11.5	12.5	13.5
## 41 24	Honda	Prelude	Sporty	17.0	19.8	22.7
## 42 42	Honda	Civic	Small	8.4	12.1	15.8
## 43 24	Honda	Accord	Compact	13.8	17.5	21.2
## 44 29	Hyundai	Excel	Small	6.8	8.0	9.2
## 45 22	Hyundai	Elantra	Small	9.0	10.0	11.0
## 46 26	Hyundai	Scoupe	Sporty	9.1	10.0	11.0
## 47 20	Hyundai	Sonata	Midsize	12.4	13.9	15.3
## 48 17	Infiniti	Q45	Midsize	45.4	47.9	50.4
## 49 18	Lexus	ES300	Midsize	27.5	28.0	28.4

## 50 18	Lexus	SC300	Midsize	34.7	35.2	35.6
## 51 17	Lincoln	Continental	Midsize	33.3	34.3	35.3
## 52 18	Lincoln	Town_Car	Large	34.4	36.1	37.8
## 53 29	Mazda	323	Small	7.4	8.3	9.1
## 54 28	Mazda	Protege	Small	10.9	11.6	12.3
## 55 26	Mazda	626	Compact	14.3	16.5	18.7
## 58 20	Mercedes-Benz	190E	Compact	29.0	31.9	34.9
## 59 19	Mercedes-Benz	300E	Midsize	43.8	61.9	80.0
## 60 23	Mercury	Capri	Sporty	13.3	14.1	15.0
## 61 19	Mercury	Cougar	Midsize	14.9	14.9	14.9
## 62 29	Mitsubishi	Mirage	Small	7.7	10.3	12.9
## 63 18	Mitsubishi	Diamante	Midsize	22.4	26.1	29.9
## 64 29	Nissan	Sentra	Small	8.7	11.8	14.9
## 65 24	Nissan	Altima	Compact	13.0	15.7	18.3
## 67 21	Nissan	Maxima	Midsize	21.0	21.5	22.0
## 68 24	Oldsmobile	Achieva	Compact	13.0	13.5	14.0
## 69 23	Oldsmobile	Cutlass_Ciera	Midsize	14.2	16.3	18.4
## 71 19	Oldsmobile	Eighty-Eight	Large	19.5	20.7	21.9
## 72 23	Plymouth	Laser	Sporty	11.4	14.4	17.4
## 73 31	Pontiac	LeMans	Small	8.2	9.0	9.9
## 74 23	Pontiac	Sunbird	Compact	9.4	11.1	12.8
## 75 19	Pontiac	Firebird	Sporty	14.0	17.7	21.4
## 76 19	Pontiac	Grand_Prix	Midsize	15.4	18.5	21.6
## 77 19	Pontiac	Bonneville	Large	19.4	24.4	29.4
## 78 20	Saab	900	Compact	20.3	28.7	37.1

## 79	Saturn	SL	Small	9.2	11.1	12.9
28						
## 80	Subaru	Justy	Small	7.3	8.4	9.5
33						
## 81	Subaru	Loyale	Small	10.5	10.9	11.3
25						
## 82	Subaru	Legacy	Compact	16.3	19.5	22.7
23						
## 83	Suzuki	Swift	Small	7.3	8.6	10.0
39						
## 84	Toyota	Tercel	Small	7.8	9.8	11.8
32						
## 85	Toyota	Celica	Sporty	14.2	18.4	22.6
25						
## 86	Toyota	Camry	Midsize	15.2	18.2	21.2
22						
## 88	Volkswagen	Fox	Small	8.7	9.1	9.5
25						
## 90	Volkswagen	Passat	Compact	17.6	20.0	22.4
21						
## 91	Volkswagen	Corrado	Sporty	22.9	23.3	23.7
18						
## 92	Volvo	240	Compact	21.8	22.7	23.5
21						
## 93	Volvo	850	Midsize	24.8	26.7	28.5
20						
##	MPG.highway	AirBags	DriveTrain	Cylinders	EngineSize	Ho
rsepower						
## 1	31	None	Front	4	1.8	
140						
## 2	25	Driver & Passenger	Front	6	3.2	
200						
## 3	26	Driver only	Front	6	2.8	
172						
## 4	26	Driver & Passenger	Front	6	2.8	
172						
## 5	30	Driver only	Rear	4	3.5	
208						
## 6	31	Driver only	Front	4	2.2	
110						
## 7	28	Driver only	Front	6	3.8	
170						
## 8	25	Driver only	Rear	6	5.7	
180						
## 9	27	Driver only	Front	6	3.8	
170						
## 10	25	Driver only	Front	8	4.9	
200						
## 11	25	Driver & Passenger	Front	8	4.6	
295						

## 12	36	None	Front	4	2.2
110					
## 13	34	Driver only	Front	4	2.2
110					
## 14	28	Driver & Passenger	Rear	6	3.4
160					
## 15	29	None	Front	4	2.2
110					
## 18	26	Driver only	Rear	8	5.0
170					
## 20	28	Driver & Passenger	Front	6	3.3
153					
## 21	28	Driver & Passenger	Front	4	3.0
141					
## 22	26	Driver only	Front	6	3.3
147					
## 23	33	None	Front	4	1.5
92					
## 24	29	Driver only	Front	4	2.2
93					
## 25	27	Driver only	Front	4	2.5
100					
## 27	27	Driver only	Front	4	2.5
100					
## 28	24	Driver only	4WD	6	3.0
300					
## 29	33	None	Front	4	1.5
92					
## 30	28	Driver & Passenger	Front	6	3.5
214					
## 31	33	None	Front	4	1.3
63					
## 32	30	None	Front	4	1.8
127					
## 33	27	None	Front	4	2.3
96					
## 34	29	Driver only	Rear	4	2.3
105					
## 35	30	Driver only	Front	4	2.0
115					
## 37	30	Driver only	Front	6	3.0
140					
## 38	26	Driver only	Rear	8	4.6
190					
## 39	50	None	Front	3	1.0
55					
## 40	36	Driver only	Front	4	1.6
90					
## 41	31	Driver & Passenger	Front	4	2.3
160					

## 42	46	Driver only	Front	4	1.5
102					
## 43	31	Driver & Passenger	Front	4	2.2
140					
## 44	33	None	Front	4	1.5
81					
## 45	29	None	Front	4	1.8
124					
## 46	34	None	Front	4	1.5
92					
## 47	27	None	Front	4	2.0
128					
## 48	22	Driver only	Rear	8	4.5
278					
## 49	24	Driver only	Front	6	3.0
185					
## 50	23	Driver & Passenger	Rear	6	3.0
225					
## 51	26	Driver & Passenger	Front	6	3.8
160					
## 52	26	Driver & Passenger	Rear	8	4.6
210					
## 53	37	None	Front	4	1.6
82					
## 54	36	None	Front	4	1.8
103					
## 55	34	Driver only	Front	4	2.5
164					
## 58	29	Driver only	Rear	4	2.3
130					
## 59	25	Driver & Passenger	Rear	6	3.2
217					
## 60	26	Driver only	Front	4	1.6
100					
## 61	26	None	Rear	6	3.8
140					
## 62	33	None	Front	4	1.5
92					
## 63	24	Driver only	Front	6	3.0
202					
## 64	33	Driver only	Front	4	1.6
110					
## 65	30	Driver only	Front	4	2.4
150					
## 67	26	Driver only	Front	6	3.0
160					
## 68	31	None	Front	4	2.3
155					
## 69	31	Driver only	Front	4	2.2
110					

## 71	28	Driver only	Front	6	3.8
170					
## 72	30	None	4WD	4	1.8
92					
## 73	41	None	Front	4	1.6
74					
## 74	31	None	Front	4	2.0
110					
## 75	28	Driver & Passenger	Rear	6	3.4
160					
## 76	27	None	Front	6	3.4
200					
## 77	28	Driver & Passenger	Front	6	3.8
170					
## 78	26	Driver only	Front	4	2.1
140					
## 79	38	Driver only	Front	4	1.9
85					
## 80	37	None	4WD	3	1.2
73					
## 81	30	None	4WD	4	1.8
90					
## 82	30	Driver only	4WD	4	2.2
130					
## 83	43	None	Front	3	1.3
70					
## 84	37	Driver only	Front	4	1.5
82					
## 85	32	Driver only	Front	4	2.2
135					
## 86	29	Driver only	Front	4	2.2
130					
## 88	33	None	Front	4	1.8
81					
## 90	30	None	Front	4	2.0
134					
## 91	25	None	Front	6	2.8
178					
## 92	28	Driver only	Rear	4	2.3
114					
## 93	28	Driver & Passenger	Front	5	2.4
168					
##	RPM	Rev.per.mile	Man.trans.avail	Fuel.tank.capacity	Passengers
length					
## 1	6300	2890	Yes	13.2	5
177					
## 2	5500	2335	Yes	18.0	5
195					
## 3	5500	2280	Yes	16.9	5
180					

## 4	5500	2535	Yes	21.1	6
193					
## 5	5700	2545	Yes	21.1	4
186					
## 6	5200	2565	No	16.4	6
189					
## 7	4800	1570	No	18.0	6
200					
## 8	4000	1320	No	23.0	6
216					
## 9	4800	1690	No	18.8	5
198					
## 10	4100	1510	No	18.0	6
206					
## 11	6000	1985	No	20.0	5
204					
## 12	5200	2380	Yes	15.2	5
182					
## 13	5200	2665	Yes	15.6	5
184					
## 14	4600	1805	Yes	15.5	4
193					
## 15	5200	2595	No	16.5	6
198					
## 18	4200	1350	No	23.0	6
214					
## 20	5300	1990	No	18.0	6
203					
## 21	5000	2090	No	16.0	6
183					
## 22	4800	1785	No	16.0	6
203					
## 23	6000	3285	Yes	13.2	5
174					
## 24	4800	2595	Yes	14.0	5
172					
## 25	4800	2535	Yes	16.0	6
181					
## 27	4800	2465	No	16.0	6
192					
## 28	6000	2120	Yes	19.8	4
180					
## 29	6000	2505	Yes	13.2	5
174					
## 30	5800	1980	No	18.0	6
202					
## 31	5000	3150	Yes	10.0	4
141					
## 32	6500	2410	Yes	13.2	5
171					

## 33 4200 177	2805	Yes	15.9	5
## 34 4600 180	2285	Yes	15.4	4
## 35 5500 179	2340	Yes	15.5	4
## 37 4800 192	1885	No	16.0	5
## 38 4200 212	1415	No	20.0	6
## 39 5700 151	3755	Yes	10.6	4
## 40 5400 164	3250	Yes	12.4	4
## 41 5800 175	2855	Yes	15.9	4
## 42 5900 173	2650	Yes	11.9	4
## 43 5600 185	2610	Yes	17.0	4
## 44 5500 168	2710	Yes	11.9	5
## 45 6000 172	2745	Yes	13.7	5
## 46 5550 166	2540	Yes	11.9	4
## 47 6000 184	2335	Yes	17.2	5
## 48 6000 200	1955	No	22.5	5
## 49 5200 188	2325	Yes	18.5	5
## 50 6000 191	2510	Yes	20.6	4
## 51 4400 205	1835	No	18.4	6
## 52 4600 219	1840	No	20.0	6
## 53 5000 164	2370	Yes	13.2	4
## 54 5500 172	2220	Yes	14.5	5
## 55 5600 184	2505	Yes	15.5	5
## 58 5100 175	2425	Yes	14.5	5
## 59 5500 187	2220	No	18.5	5
## 60 5750 166	2475	Yes	11.1	4

## 61 3800 199	1730	No	18.0	5
## 62 6000 172	2505	Yes	13.2	5
## 63 6000 190	2210	No	19.0	5
## 64 6000 170	2435	Yes	13.2	5
## 65 5600 181	2130	Yes	15.9	5
## 67 5200 188	2045	No	18.5	5
## 68 6000 188	2380	No	15.2	5
## 69 5200 190	2565	No	16.5	5
## 71 4800 201	1570	No	18.0	6
## 72 5000 173	2360	Yes	15.9	4
## 73 5600 177	3130	Yes	13.2	4
## 74 5200 181	2665	Yes	15.2	5
## 75 4600 196	1805	Yes	15.5	4
## 76 5000 195	1890	Yes	16.5	5
## 77 4800 177	1565	No	18.0	6
## 78 6000 184	2910	Yes	18.0	5
## 79 5000 176	2145	Yes	12.8	5
## 80 5600 146	2875	Yes	9.2	4
## 81 5200 175	3375	Yes	15.9	5
## 82 5600 179	2330	Yes	15.9	5
## 83 6000 161	3360	Yes	10.6	4
## 84 5200 162	3505	Yes	11.9	5
## 85 5400 174	2405	Yes	15.9	4
## 86 5400 188	2340	Yes	18.5	5
## 88 5500 163	2550	Yes	12.4	4

## 90	5800	2685	Yes	18.5	5	
180						
## 91	5800	2385	Yes	18.5	4	
159						
## 92	5400	2215	Yes	15.8	5	
190						
## 93	6200	2310	Yes	19.3	5	
184						
##	Wheelbase	Width	Turn.circle	Rear.seat.room	Luggage.room	Weight 0
rigin						
## 1	102	68	37	26.5	11	2705 no
n-USA						
## 2	115	71	38	30.0	15	3560 no
n-USA						
## 3	102	67	37	28.0	14	3375 no
n-USA						
## 4	106	70	37	31.0	17	3405 no
n-USA						
## 5	109	69	39	27.0	13	3640 no
n-USA						
## 6	105	69	41	28.0	16	2880
USA						
## 7	111	74	42	30.5	17	3470
USA						
## 8	116	78	45	30.5	21	4105
USA						
## 9	108	73	41	26.5	14	3495
USA						
## 10	114	73	43	35.0	18	3620
USA						
## 11	111	74	44	31.0	14	3935
USA						
## 12	101	66	38	25.0	13	2490
USA						
## 13	103	68	39	26.0	14	2785
USA						
## 14	101	74	43	25.0	13	3240
USA						
## 15	108	71	40	28.5	16	3195
USA						
## 18	116	77	42	29.5	20	3910
USA						
## 20	113	74	40	31.0	15	3515
USA						
## 21	104	68	41	30.5	14	3085
USA						
## 22	110	69	44	36.0	17	3570
USA						
## 23	98	66	32	26.5	11	2270
USA						

## 24	97	67	38	26.5	13	2670	
USA							
## 25	104	68	39	30.5	14	2970	
USA							
## 27	105	69	42	30.5	16	3080	
USA							
## 28	97	72	40	20.0	11	3805	
USA							
## 29	98	66	36	26.5	11	2295	
USA							
## 30	113	74	40	30.0	15	3490	
USA							
## 31	90	63	33	26.0	12	1845	
USA							
## 32	98	67	36	28.0	12	2530	
USA							
## 33	100	68	39	27.5	13	2690	
USA							
## 34	101	68	40	24.0	12	2850	
USA							
## 35	103	70	38	23.0	18	2710	
USA							
## 37	106	71	40	27.5	18	3325	
USA							
## 38	114	78	43	30.0	21	3950	
USA							
## 39	93	63	34	27.5	10	1695	no
n-USA							
## 40	97	67	37	24.5	11	2475	no
n-USA							
## 41	100	70	39	23.5	8	2865	no
n-USA							
## 42	103	67	36	28.0	12	2350	no
n-USA							
## 43	107	67	41	28.0	14	3040	no
n-USA							
## 44	94	63	35	26.0	11	2345	no
n-USA							
## 45	98	66	36	28.0	12	2620	no
n-USA							
## 46	94	64	34	23.5	9	2285	no
n-USA							
## 47	104	69	41	31.0	14	2885	no
n-USA							
## 48	113	72	42	29.0	15	4000	no
n-USA							
## 49	103	70	40	27.5	14	3510	no
n-USA							
## 50	106	71	39	25.0	9	3515	no
n-USA							

## 51 USA	109	73	42	30.0	19	3695	
## 52 USA	117	77	45	31.5	22	4055	
## 53 n-USA	97	66	34	27.0	16	2325	no
## 54 n-USA	98	66	36	26.5	13	2440	no
## 55 n-USA	103	69	40	29.5	14	2970	no
## 58 n-USA	105	67	34	26.0	12	2920	no
## 59 n-USA	110	69	37	27.0	15	3525	no
## 60 USA	95	65	36	19.0	6	2450	
## 61 USA	113	73	38	28.0	15	3610	
## 62 n-USA	98	67	36	26.0	11	2295	no
## 63 n-USA	107	70	43	27.5	14	3730	no
## 64 n-USA	96	66	33	26.0	12	2545	no
## 65 n-USA	103	67	40	28.5	14	3050	no
## 67 n-USA	104	69	41	28.5	14	3200	no
## 68 USA	103	67	39	28.0	14	2910	
## 69 USA	105	70	42	28.0	16	2890	
## 71 USA	111	74	42	31.5	17	3470	
## 72 USA	97	67	39	24.5	8	2640	
## 73 USA	99	66	35	25.5	17	2350	
## 74 USA	101	66	39	25.0	13	2575	
## 75 USA	101	75	43	25.0	13	3240	
## 76 USA	108	72	41	28.5	16	3450	
## 77 USA	111	74	43	30.5	18	3495	
## 78 n-USA	99	67	37	26.5	14	2775	no
## 79 USA	102	68	40	26.5	12	2495	

## 80	90	60	32	23.5	10	2045	no
n-USA							
## 81	97	65	35	27.5	15	2490	no
n-USA							
## 82	102	67	37	27.0	14	3085	no
n-USA							
## 83	93	63	34	27.5	10	1965	no
n-USA							
## 84	94	65	36	24.0	11	2055	no
n-USA							
## 85	99	69	39	23.0	13	2950	no
n-USA							
## 86	103	70	38	28.5	15	3030	no
n-USA							
## 88	93	63	34	26.0	10	2240	no
n-USA							
## 90	103	67	35	31.5	14	2985	no
n-USA							
## 91	97	66	36	26.0	15	2810	no
n-USA							
## 92	104	67	37	29.5	14	2985	no
n-USA							
## 93	105	69	38	30.0	15	3245	no
n-USA							
##			Make				
## 1			Acura Integra				
## 2			Acura Legend				
## 3			Audi 90				
## 4			Audi 100				
## 5			BMW 535i				
## 6			Buick Century				
## 7			Buick LeSabre				
## 8			Buick Roadmaster				
## 9			Buick Riviera				
## 10			Cadillac DeVille				
## 11			Cadillac Seville				
## 12			Chevrolet Cavalier				
## 13			Chevrolet Corsica				
## 14			Chevrolet Camaro				
## 15			Chevrolet Lumina				
## 18			Chevrolet Caprice				
## 20			Chrysler Concorde				
## 21			Chrysler LeBaron				
## 22			Chrysler Imperial				
## 23			Dodge Colt				
## 24			Dodge Shadow				
## 25			Dodge Spirit				
## 27			Dodge Dynasty				
## 28			Dodge Stealth				
## 29			Eagle Summit				

## 30	Eagle Vision
## 31	Ford Festiva
## 32	Ford Escort
## 33	Ford Tempo
## 34	Ford Mustang
## 35	Ford Probe
## 37	Ford Taurus
## 38	Ford Crown_Victoria
## 39	Geo Metro
## 40	Geo Storm
## 41	Honda Prelude
## 42	Honda Civic
## 43	Honda Accord
## 44	Hyundai Excel
## 45	Hyundai Elantra
## 46	Hyundai Scoupe
## 47	Hyundai Sonata
## 48	Infiniti Q45
## 49	Lexus ES300
## 50	Lexus SC300
## 51	Lincoln Continental
## 52	Lincoln Town_Car
## 53	Mazda 323
## 54	Mazda Protege
## 55	Mazda 626
## 58	Mercedes-Benz 190E
## 59	Mercedes-Benz 300E
## 60	Mercury Capri
## 61	Mercury Cougar
## 62	Mitsubishi Mirage
## 63	Mitsubishi Diamante
## 64	Nissan Sentra
## 65	Nissan Altima
## 67	Nissan Maxima
## 68	Oldsmobile Achieva
## 69	Oldsmobile Cutlass_Ciera
## 71	Oldsmobile Eighty-Eight
## 72	Plymouth Laser
## 73	Pontiac LeMans
## 74	Pontiac Sunbird
## 75	Pontiac Firebird
## 76	Pontiac Grand_Prix
## 77	Pontiac Bonneville
## 78	Saab 900
## 79	Saturn SL
## 80	Subaru Justy
## 81	Subaru Loyale
## 82	Subaru Legacy
## 83	Suzuki Swift
## 84	Toyota Tercel

```
## 85          Toyota Celica
## 86          Toyota Camry
## 88          Volkswagen Fox
## 90          Volkswagen Passat
## 91          Volkswagen Corrado
## 92          Volvo 240
## 93          Volvo 850
```

填補遺漏值

方法一：平均數

```
df$Luggage.room[is.na(df$Luggage.room)] <- mean(df$Luggage.room, na.rm
= TRUE)
colSums(is.na(df)) # Luggage.room 欄位已經沒有遺漏值
```

```
##      Manufacturer      Model      Type      Mi
n.Price
##      0      0      0
0
##      Price      Max.Price      MPG.city      MPG.
highway
##      0      0      0
0
##      AirBags      DriveTrain      Cylinders      Eng
ineSize
##      0      0      0
0
##      Horsepower      RPM      Rev.per.mile      Man.tran
s.avail
##      0      0      0
0
##      Fuel.tank.capacity      Passengers      Length      Wh
eelbase
##      0      0      0
0
##      Width      Turn.circle      Rear.seat.room      Lugga
ge.room
##      0      0      2
0
##      Weight      Origin      Make
##      0      0      0
```

方法二：中位數

```
df$Rear.seat.room <- median(df$Rear.seat.room, na.rm = TRUE)
sum(is.na(df[, 'Rear.seat.room']))

## [1] 0
```

方法三：固定值

```
# replace_na {tidyr}
```

2.重複值(Duplicate Records)

範例：

Bob 有重複。

ID	Name	Gender	Math	English
1	ALAN	F	95	80
2	BOB	M	70	85
2	BOB	M	70	85
4	DAVID	M	90	77

```
df <- data.frame(  
  ID      = c(1, 2, 2, 4),  
  Name    = c("ALAN", "BOB", "BOB", "DAVID"),  
  Gender  = c("F", "M", "M", "M"),  
  Math    = c(95, 70, 70, 90),  
  English = c(80, 85, 85, 77)  
)  
df
```

```
##   ID Name Gender Math English  
## 1  1  ALAN      F   95      80  
## 2  2   BOB      M   70      85  
## 3  2   BOB      M   70      85  
## 4  4 DAVID      M   90      77
```

找出重複資料列

```
duplicated(df)
```

```
## [1] FALSE FALSE  TRUE FALSE
```

查看重複資料

```
df[duplicated(df), ]
```

```
##   ID Name Gender Math English  
## 3  2  BOB      M   70      85
```

刪除重複資料列

```
unique(df)
```

```
##   ID Name Gender Math English  
## 1  1  ALAN      F   95      80
```

```
## 2 2 BOB M 70 85
## 4 4 DAVID M 90 77
```

刪除重複資料列 {dplyr}

```
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
## filter, lag

## The following objects are masked from 'package:base':
##
## intersect, setdiff, setequal, union

distinct(df)

##   ID Name Gender Math English
## 1  1 ALAN      F   95      80
## 2  2 BOB      M   70      85
## 3  4 DAVID     M   90      77
```

3.資料型態錯誤(Incorrect Data Types)

常見資料型態錯誤

問題	常見原因	解決方式
數值變成文字 (character)	Excel 混入文字、單位	as.numeric()
日期變成文字	CSV 匯入	as.Date()
類別變成文字	性別、縣市	factor()
一欄存多個資訊	“男,25”	separate()
多欄代表同一變數	Jan、Feb、Mar	pivot_longer()
一欄混合不同型態	“100kg”	去除單位再轉型
缺失值表示方式不一致	“,”-“,”NA”	統一成 NA
重複資料	匯入重覆	distinct()
欄名不一致	Age、AGE、age	rename()

- 字串轉換為數值: `df$Column <- as.numeric(df$Column)`
- 數值轉換為字串: `df$Column <- as.character(df$Column)`
- 字串轉換為日期: `df$Column <- as.Date(df$Column)`

字串轉換為日期

```
tmp <- as.Date("31/08/2026", format="%d/%m/%Y")
print(tmp)

## [1] "2026-08-31"

class(tmp)

## [1] "Date"
```

字串轉換為因子 (Factor)

```
# df$Column <- as.factor(df$Column)
```

因子轉換字串

```
# df$Column <- as.character(df$Column)
```

🌸 2.2 dplyr :

filter/select/mutate/summarise/group_by

管線運算子(%>%)

- magrittr 套件中主要的功能是提供 %>% 管線運算子(Pipe Operator)
- 管線運算子使用 ‘|>’ 或 ‘%>%’ 是一種將前一個指令或函數的輸出結果 (output), 直接傳遞給下一個函數作為輸入的工具(input), 即做為右側函數的第一個參數輸入.
- 它能让程式碼由左至右、由上而下順暢閱讀, 提升可讀性避免多層巢狀括號的混亂.
- 管線運算子能減少暫存變數, 不需要為每一個中間步驟宣告額外的變數.
- RStudio Global Options Tools Code Editing: [Use native pipe operator,|> (requires R 4.1+)] 打勾
- CRAN magrittr 套件: <https://cran.r-project.org/web/packages/magrittr/index.html>
- magrittr 套件簡介: <https://cran.r-project.org/web/packages/magrittr/vignettes/magrittr.html>
- magrittr 套件範例

```
# 1 加侖 = 3.785 公升
# 1 公里 = 0.621371192 英里
```

```

# 原資料: miles per gallon
# 新資料: kilometers per liter
# 新資料 = miles per gallon*((1/0.621371192)/3.785)=0.42519

library(magrittr)

car_data <-
  mtcars %>%
  subset(hp > 100) %>%
  aggregate(. ~ cyl, data = ., FUN = . %>% mean %>% round(2)) %>%
  transform(kpl = mpg %>% multiply_by(0.4251)) %>%
  print

##   cyl   mpg   disp  hp drat   wt  qsec    vs  am gear carb
## 1    4 25.90 108.05 111.00 3.94 2.15 17.75 1.00 1.00 4.50 2.00 11.010
## 2    6 19.74 183.31 122.29 3.59 3.12 17.98 0.57 0.43 3.86 3.43  8.391
## 3    8 15.10 353.10 209.21 3.23 4.00 16.77 0.00 0.14 3.29 3.50  6.419

```

認識 dplyr 套件

- r cran dplyr: <https://cran.r-project.org/web/packages/dplyr/index.html>
- dplyr: A Grammar of Data Manipulation 資料操作文法
- dplyr = data frame + plyr
- plyr 發音 plier (鉗子), plyr 是 dplyr 的前個套件版本, 作者推薦使用 dplyr 套件
- plyr: <https://www.slideshare.net/hadley/plyr-one-data-analytic-strategy>

flights 2013 年 NYC 機場航班資料, 33 萬筆資料

```

library(dplyr)
library(nycflights13)

data(flights) # tibble: 336776*19
print(flights)

## # A tibble: 336,776 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sche
d_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
##   <int>
## 1  2013     1     1     517             515           2     830
##   819
## 2  2013     1     1     533             529           4     850
##   830
## 3  2013     1     1     542             540           2     923
##   850

```



```
## 4 2013 1 1 544 545 -1 1004
1022
## 5 2013 1 1 554 600 -6 812
837
## 6 2013 1 1 554 558 -4 740
728
## 7 2013 1 1 555 600 -5 913
854
## 8 2013 1 1 557 600 -3 709
723
## 9 2013 1 1 557 600 -3 838
846
## 10 2013 1 1 558 600 -2 753
745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## # tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## # hour <dbl>, minute <dbl>, time_hour <dtm>

class(flights) # "tbl_df" "tbl" "data.frame"

## [1] "tbl_df" "tbl" "data.frame"
```

- flights 資料集變數說明

變數	資料型態	中文說明	範例
year	integer	年份	2013
month	integer	月份	1~12
day	integer	日期	1~31
dep_time	integer	實際起飛時間(HHMM)	517(表示 05:17), NA 取消
sched_dep_time	integer	表定起飛時間(HHMM)	515
dep_delay	double	起飛延誤(分鐘)	2、-5、60
arr_time	integer	實際抵達時間(HHMM)	830
sched_arr_time	integer	表定抵達時間(HHMM)	819
arr_delay	double	抵達延誤(分鐘)	11、-20
carrier	character	航空公司代碼	AA、DL
flight	integer	航班編號	1545
tailnum	character	飛機註冊編號	N14228
origin	character	起飛機場	JFK
dest	character	目的地機場	LAX
air_time	double	飛行時間(分鐘)	150
distance	double	飛行距離(英里)	950

變數	資料型態	中文說明	範例
hour	double	表定起飛小時	5
min	double	表定起飛分鐘	15
time_hour	POSIXct	日期時間(UTC)	2013-01-01 05:00

- dep_delay:
 - dep_delay 起飛延誤 = 實際起飛 - 表定起飛
 - -10: 提早 10 分鐘
 - 0: 準時
 - 15: 延誤 15 分鐘
 - 120: 延誤 2 小時
- arr_delay:
 - arr_delay 抵達延誤 = 實際抵達 - 表定抵達, 抵達延誤可用於分析航班績效重要的變數
 - 11: 晚 11 分鐘抵達
 - -20: 提早 20 分鐘抵達
- 資料檢視函數,快速查看資料框 (data.frame 或 tibble) 的結構.

```
glimpse(flights)
## Rows: 336,776
## Columns: 19
## $ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 201
3, 2013, 2...
## $ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1...
## $ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1...
## $ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557,
558, 558, ...
## $ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600,
600, 600, ...
## $ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2,
-2, -2, -1...
## $ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838,
753, 849,...
## $ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846,
745, 851,...
## $ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2,
```

```

-3, 7, -1...
## $ carrier      <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV
", "B6", "...
## $ flight      <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708,
79, 301, 4...
## $ tailnum     <chr> "N14228", "N24211", "N619AA", "N804JB", "N668
DN", "N394...
## $ origin      <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EW
R", "LGA",...
## $ dest        <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FL
L", "IAD",...
## $ air_time    <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 1
38, 149, 1...
## $ distance    <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229,
944, 733, ...
## $ hour        <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6,
5, 6, 6, 6...
## $ minute      <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0,
0, 59, 0...
## $ time_hour   <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 20
13-01-01 0...

```

- 資料結構

```

str(flights)

## tibble [336,776 × 19] (S3: tbl_df/tbl/data.frame)
## $ year         : int [1:336776] 2013 2013 2013 2013 2013 2013 2013
2013 2013 2013 ...
## $ month        : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
## $ day          : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
## $ dep_time     : int [1:336776] 517 533 542 544 554 554 555 557 55
7 558 ...
## $ sched_dep_time: int [1:336776] 515 529 540 545 600 558 600 600 60
0 600 ...
## $ dep_delay    : num [1:336776] 2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
## $ arr_time     : int [1:336776] 830 850 923 1004 812 740 913 709 8
38 753 ...
## $ sched_arr_time: int [1:336776] 819 830 850 1022 837 728 854 723 8
46 745 ...
## $ arr_delay    : num [1:336776] 11 20 33 -18 -25 12 19 -14 -8 8 ...
## $ carrier      : chr [1:336776] "UA" "UA" "AA" "B6" ...
## $ flight       : int [1:336776] 1545 1714 1141 725 461 1696 507 57
08 79 301 ...
## $ tailnum      : chr [1:336776] "N14228" "N24211" "N619AA" "N804JB
" ...
## $ origin       : chr [1:336776] "EWR" "LGA" "JFK" "JFK" ...
## $ dest         : chr [1:336776] "IAH" "IAH" "MIA" "BQN" ...
## $ air_time     : num [1:336776] 227 227 160 183 116 150 158 53 140
138 ...
## $ distance     : num [1:336776] 1400 1416 1089 1576 762 ...

```

```
## $ hour      : num [1:336776] 5 5 5 5 6 5 6 6 6 6 ...
## $ minute    : num [1:336776] 15 29 40 45 0 58 0 0 0 0 ...
## $ time_hour : POSIXct[1:336776], format: "2013-01-01 05:00:00"
"2013-01-01 05:00:00" ...
```

- 資料摘要

```
summary(flights) # 有NA
```

```
##      year      month      day      dep_time      sche
d_dep_time
## Min.      :2013    Min.      : 1.000    Min.      : 1.00    Min.      : 1    Min.
: 106
## 1st Qu.:2013    1st Qu.: 4.000    1st Qu.: 8.00    1st Qu.: 907    1st
Qu.: 906
## Median :2013    Median : 7.000    Median :16.00    Median :1401    Medi
an :1359
## Mean      :2013    Mean      : 6.549    Mean      :15.71    Mean      :1349    Mean
:1344
## 3rd Qu.:2013    3rd Qu.:10.000    3rd Qu.:23.00    3rd Qu.:1744    3rd
Qu.:1729
## Max.      :2013    Max.      :12.000    Max.      :31.00    Max.      :2400    Max.
:2359
##
##                                     NAs      :8255

##      dep_delay      arr_time      sched_arr_time      arr_delay
## Min.      : -43.00    Min.      : 1    Min.      : 1    Min.      : -86.000
## 1st Qu.: -5.00    1st Qu.:1104    1st Qu.:1124    1st Qu.: -17.000
## Median : -2.00    Median :1535    Median :1556    Median : -5.000
## Mean      : 12.64    Mean      :1502    Mean      :1536    Mean      : 6.895
## 3rd Qu.: 11.00    3rd Qu.:1940    3rd Qu.:1945    3rd Qu.: 14.000
## Max.      :1301.00    Max.      :2400    Max.      :2359    Max.      :1272.000
## NAs      :8255    NAs      :8713    NAs      :9430
##      carrier      flight      tailnum      origin
## Length      :336776    Min.      : 1    Length      :336776    Length      :3367
76
## N.unique : 16    1st Qu.: 553    N.unique : 4043    N.unique :
3
## N.blank : 0    Median :1496    N.blank : 0    N.blank :
0
## Min.nchar: 2    Mean      :1972    Min.nchar: 5    Min.nchar:
3
## Max.nchar: 2    3rd Qu.:3465    Max.nchar: 6    Max.nchar:
3
##
##      Max.      :8500    NAs      : 2512

##

##      dest      air_time      distance      hour
```

```
## Length :336776 Min. : 20.0 Min. : 17 Min. : 1.00
## N.unique : 105 1st Qu.: 82.0 1st Qu.: 502 1st Qu.: 9.00
## N.blank : 0 Median :129.0 Median : 872 Median :13.00
## Min.nchar: 3 Mean :150.7 Mean :1040 Mean :13.18
## Max.nchar: 3 3rd Qu.:192.0 3rd Qu.:1389 3rd Qu.:17.00
## Max. :695.0 Max. :4983 Max. :23.00
## NAs :9430
## minute time_hour
## Min. : 0.00 Min. :2013-01-01 05:00:00
## 1st Qu.: 8.00 1st Qu.:2013-04-04 13:00:00
## Median :29.00 Median :2013-07-03 10:00:00
## Mean :26.23 Mean :2013-07-03 05:22:54
## 3rd Qu.:44.00 3rd Qu.:2013-10-01 07:00:00
## Max. :59.00 Max. :2013-12-31 23:00:00
##
```

- 如何轉換為 `tbl_df`, 使用 `tibble::as.tbl`

```
mytbl <- tibble::as_tibble(iris)
class(mytbl)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

- 前 6 筆資料

```
head(flights)
```

```
## # A tibble: 6 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched
 _arr_time
##   <int> <int> <int>   <int>         <int>      <dbl>   <int>
##   <int>
## 1  2013     1     1     517             515         2     830
##     819
## 2  2013     1     1     533             529         4     850
##     830
## 3  2013     1     1     542             540         2     923
##     850
## 4  2013     1     1     544             545        -1    1004
##    1022
## 5  2013     1     1     554             600        -6     812
##     837
## 6  2013     1     1     554             558        -4     740
##     728
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- 後 6 筆資料

注意: 資料最後日期為 12 月 31 日, 但資料排序不是依照月, 日排序, 最後一筆是 9 月 30 日

```
tail(flights)
```

```
## # A tibble: 6 × 19
```

```
##   year month   day dep_time sched_dep_time dep_delay arr_time sched  
_arr_time
```

```
##   <int> <int> <int>   <int>         <int>      <dbl>   <int>  
   <int>
```

```
## 1  2013     9    30      NA         1842      NA      NA  
    2019
```

```
## 2  2013     9    30      NA         1455      NA      NA  
    1634
```

```
## 3  2013     9    30      NA         2200      NA      NA  
    2312
```

```
## 4  2013     9    30      NA         1210      NA      NA  
    1330
```

```
## 5  2013     9    30      NA         1159      NA      NA  
    1344
```

```
## 6  2013     9    30      NA          840      NA      NA  
    1020
```

```
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,  
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc  
e <dbl>,  
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

dplyr 套件重要函數

函數	說明
filter	條件式篩選資料
slice	列的指標篩選資料
arrange	排序
select	選取行/更改欄位名稱
rename	選取所有行/更改欄位名稱
distinct	選取不重覆資料
mutate	新增欄位, 保留原資料
transmute	新增欄位, 不保留原資料
summarise	群組計算

filter 條件式篩選資料

```
filter(flights, month == 1, day == 1)
```

```
## # A tibble: 842 × 19
```

```
##   year month   day dep_time sched_dep_time dep_delay arr_time sche  
d_arr_time
```

```

##      <int> <int> <int>      <int>      <int>      <dbl>      <int>
##      <int>
## 1  2013      1      1      517      515      2      830
##      819
## 2  2013      1      1      533      529      4      850
##      830
## 3  2013      1      1      542      540      2      923
##      850
## 4  2013      1      1      544      545     -1     1004
##     1022
## 5  2013      1      1      554      600     -6      812
##      837
## 6  2013      1      1      554      558     -4      740
##      728
## 7  2013      1      1      555      600     -5      913
##      854
## 8  2013      1      1      557      600     -3      709
##      723
## 9  2013      1      1      557      600     -3      838
##      846
## 10 2013      1      1      558      600     -2      753
##      745
## # i 832 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

flights[flights$month == 1 & flights$day == 1, ] # 基本指令, same as fil
ter

## # A tibble: 842 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sche
##   <int> <int> <int>   <int>      <int>      <dbl>      <int>
##   <int>
## 1  2013      1      1      517      515      2      830
##      819
## 2  2013      1      1      533      529      4      850
##      830
## 3  2013      1      1      542      540      2      923
##      850
## 4  2013      1      1      544      545     -1     1004
##     1022
## 5  2013      1      1      554      600     -6      812
##      837
## 6  2013      1      1      554      558     -4      740
##      728
## 7  2013      1      1      555      600     -5      913
##      854

```

```

## 8 2013 1 1 557 600 -3 709
723
## 9 2013 1 1 557 600 -3 838
846
## 10 2013 1 1 558 600 -2 753
745
## # i 832 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## # tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
e <dbl>,
## # hour <dbl>, minute <dbl>, time_hour <dtm>

filter(flights, month == 1 | month == 2) # AND 條件篩選, 同理 OR 使用 |

## # A tibble: 51,955 × 19
## year month day dep_time sched_dep_time dep_delay arr_time sche
d_arr_time
## <int> <int> <int> <int> <int> <dbl> <int>
<int>
## 1 2013 1 1 517 515 2 830
819
## 2 2013 1 1 533 529 4 850
830
## 3 2013 1 1 542 540 2 923
850
## 4 2013 1 1 544 545 -1 1004
1022
## 5 2013 1 1 554 600 -6 812
837
## 6 2013 1 1 554 558 -4 740
728
## 7 2013 1 1 555 600 -5 913
854
## 8 2013 1 1 557 600 -3 709
723
## 9 2013 1 1 557 600 -3 838
846
## 10 2013 1 1 558 600 -2 753
745
## # i 51,945 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## # tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
e <dbl>,
## # hour <dbl>, minute <dbl>, time_hour <dtm>

```

slice 列的指標篩選資料

```

# 第1筆
slice(flights, 1)

```



```
## # A tibble: 1 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched
  _arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
##   <int>
## 1  2013     1     1     517           515         2     830
##    819
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

# 最後一筆
slice(flights, n())

## # A tibble: 1 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched
  _arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
##   <int>
## 1  2013     9    30      NA           840         NA     NA
##    1020
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

# 第123456 筆至最後一筆
slice(flights, 123456:n())

## # A tibble: 213,321 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sche
  d_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
##   <int>
## 1  2013     2    14    2041           1932         69    2316
##    2207
## 2  2013     2    14    2043           2045        -2    2145
##    2216
## 3  2013     2    14    2049           2048         1    2221
##    2213
## 4  2013     2    14    2049           2048         1         4
##    2356
## 5  2013     2    14    2050           2040        10    2152
##    2148
## 6  2013     2    14    2050           2040        10    2350
##    2356
## 7  2013     2    14    2051           2020        31    2230
##    2159
## 8  2013     2    14    2052           1959        53    2230
```

```

      2144
## 9 2013      2    14      2052      2100      -8      2232
      2250
## 10 2013      2    14      2052      2100      -8      2227
      2235
## # i 213,311 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

```

arrange 排序

依照年,月,日遞增排序

```
arrange(flights, year, month, day)
```

```

## # A tibble: 336,776 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sche
##   <int> <int> <int>   <int>         <int>      <dbl>   <int>
##   <int>
## 1 2013      1     1     517           515         2     830
##   819
## 2 2013      1     1     533           529         4     850
##   830
## 3 2013      1     1     542           540         2     923
##   850
## 4 2013      1     1     544           545        -1    1004
##   1022
## 5 2013      1     1     554           600        -6     812
##   837
## 6 2013      1     1     554           558        -4     740
##   728
## 7 2013      1     1     555           600        -5     913
##   854
## 8 2013      1     1     557           600        -3     709
##   723
## 9 2013      1     1     557           600        -3     838
##   846
## 10 2013      1     1     558           600        -2     753
##   745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

```

依照延誤時間遞減排序

```
arrange(flights, desc(arr_delay))
```

```
## # A tibble: 336,776 × 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sche
d_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
##   <int>
## 1  2013     1     9     641           900       1301    1242
##   1530
## 2  2013     6    15    1432          1935       1137    1607
##   2120
## 3  2013     1    10    1121          1635       1126    1239
##   1810
## 4  2013     9    20    1139          1845       1014    1457
##   2210
## 5  2013     7    22     845          1600       1005    1044
##   1815
## 6  2013     4    10    1100          1900        960    1342
##   2211
## 7  2013     3    17    2321           810        911     135
##   1020
## 8  2013     7    22    2257           759        898     121
##   1026
## 9  2013    12     5     756          1700        896    1058
##   2020
## 10 2013     5     3    1133          2055        878    1250
##   2215
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distanc
## #   e <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

select 選取行

選取 3 行資料

```
select(flights, year, month, day)
```

```
## # A tibble: 336,776 × 3
##   year month   day
##   <int> <int> <int>
## 1  2013     1     1
## 2  2013     1     1
## 3  2013     1     1
## 4  2013     1     1
## 5  2013     1     1
## 6  2013     1     1
## 7  2013     1     1
## 8  2013     1     1
## 9  2013     1     1
## 10 2013     1     1
## # i 336,766 more rows
```

select 選取行, 使用冒號(:)表示連續範圍

```
select(flights, year:dep_delay)
```

```
## # A tibble: 336,776 × 6
##   year month   day dep_time sched_dep_time dep_delay
##   <int> <int> <int>   <int>         <int>         <dbl>
## 1  2013     1     1     517             515           2
## 2  2013     1     1     533             529           4
## 3  2013     1     1     542             540           2
## 4  2013     1     1     544             545          -1
## 5  2013     1     1     554             600          -6
## 6  2013     1     1     554             558          -4
## 7  2013     1     1     555             600          -5
## 8  2013     1     1     557             600          -3
## 9  2013     1     1     557             600          -3
##10  2013     1     1     558             600          -2
## # i 336,766 more rows
```

select 選取行, 使用負號(-)表示刪除

```
select(flights, -(year:day))
```

```
## # A tibble: 336,776 × 16
##   dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_del
ay carrier
##   <int>         <int>         <dbl>   <int>         <int>         <db
l> <chr>
## 1     517             515           2     830             819
11 UA
## 2     533             529           4     850             830
20 UA
## 3     542             540           2     923             850
33 AA
## 4     544             545          -1    1004            1022     -
18 B6
## 5     554             600          -6     812             837     -
25 DL
## 6     554             558          -4     740             728
12 UA
## 7     555             600          -5     913             854
19 B6
## 8     557             600          -3     709             723     -
14 EV
## 9     557             600          -3     838             846
-8 B6
##10     558             600          -2     753             745
8 AA
## # i 336,766 more rows
## # i 9 more variables: flight <int>, tailnum <chr>, origin <chr>, des
t <chr>,
```

```
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>, time_h  
our <dtm>
```

select 選取行並且更改欄位名稱

select 只選取 tailnum 1 行資料, 更名為 tail_num

```
select(flights, tail_num = tailnum)
```

```
## # A tibble: 336,776 × 1
```

```
##   tail_num
```

```
##   <chr>
```

```
## 1 N14228
```

```
## 2 N24211
```

```
## 3 N619AA
```

```
## 4 N804JB
```

```
## 5 N668DN
```

```
## 6 N39463
```

```
## 7 N516JB
```

```
## 8 N829AS
```

```
## 9 N593JB
```

```
## 10 N3ALAA
```

```
## # i 336,766 more rows
```

select 選取行, 使用 starts_with 等參數

```
iris %>% select(starts_with("Sepal"))
```

```
##   Sepal.Length Sepal.Width
```

```
## 1           5.1           3.5
```

```
## 2           4.9           3.0
```

```
## 3           4.7           3.2
```

```
## 4           4.6           3.1
```

```
## 5           5.0           3.6
```

```
## 6           5.4           3.9
```

```
## 7           4.6           3.4
```

```
## 8           5.0           3.4
```

```
## 9           4.4           2.9
```

```
## 10          4.9           3.1
```

```
## 11          5.4           3.7
```

```
## 12          4.8           3.4
```

```
## 13          4.8           3.0
```

```
## 14          4.3           3.0
```

```
## 15          5.8           4.0
```

```
## 16          5.7           4.4
```

```
## 17          5.4           3.9
```

```
## 18          5.1           3.5
```

```
## 19          5.7           3.8
```

```
## 20          5.1           3.8
```

```
## 21          5.4           3.4
```

```
## 22          5.1           3.7
```

```
## 23          4.6           3.6
```

## 24	5.1	3.3
## 25	4.8	3.4
## 26	5.0	3.0
## 27	5.0	3.4
## 28	5.2	3.5
## 29	5.2	3.4
## 30	4.7	3.2
## 31	4.8	3.1
## 32	5.4	3.4
## 33	5.2	4.1
## 34	5.5	4.2
## 35	4.9	3.1
## 36	5.0	3.2
## 37	5.5	3.5
## 38	4.9	3.6
## 39	4.4	3.0
## 40	5.1	3.4
## 41	5.0	3.5
## 42	4.5	2.3
## 43	4.4	3.2
## 44	5.0	3.5
## 45	5.1	3.8
## 46	4.8	3.0
## 47	5.1	3.8
## 48	4.6	3.2
## 49	5.3	3.7
## 50	5.0	3.3
## 51	7.0	3.2
## 52	6.4	3.2
## 53	6.9	3.1
## 54	5.5	2.3
## 55	6.5	2.8
## 56	5.7	2.8
## 57	6.3	3.3
## 58	4.9	2.4
## 59	6.6	2.9
## 60	5.2	2.7
## 61	5.0	2.0
## 62	5.9	3.0
## 63	6.0	2.2
## 64	6.1	2.9
## 65	5.6	2.9
## 66	6.7	3.1
## 67	5.6	3.0
## 68	5.8	2.7
## 69	6.2	2.2
## 70	5.6	2.5
## 71	5.9	3.2
## 72	6.1	2.8
## 73	6.3	2.5

## 74	6.1	2.8
## 75	6.4	2.9
## 76	6.6	3.0
## 77	6.8	2.8
## 78	6.7	3.0
## 79	6.0	2.9
## 80	5.7	2.6
## 81	5.5	2.4
## 82	5.5	2.4
## 83	5.8	2.7
## 84	6.0	2.7
## 85	5.4	3.0
## 86	6.0	3.4
## 87	6.7	3.1
## 88	6.3	2.3
## 89	5.6	3.0
## 90	5.5	2.5
## 91	5.5	2.6
## 92	6.1	3.0
## 93	5.8	2.6
## 94	5.0	2.3
## 95	5.6	2.7
## 96	5.7	3.0
## 97	5.7	2.9
## 98	6.2	2.9
## 99	5.1	2.5
## 100	5.7	2.8
## 101	6.3	3.3
## 102	5.8	2.7
## 103	7.1	3.0
## 104	6.3	2.9
## 105	6.5	3.0
## 106	7.6	3.0
## 107	4.9	2.5
## 108	7.3	2.9
## 109	6.7	2.5
## 110	7.2	3.6
## 111	6.5	3.2
## 112	6.4	2.7
## 113	6.8	3.0
## 114	5.7	2.5
## 115	5.8	2.8
## 116	6.4	3.2
## 117	6.5	3.0
## 118	7.7	3.8
## 119	7.7	2.6
## 120	6.0	2.2
## 121	6.9	3.2
## 122	5.6	2.8
## 123	7.7	2.8

```
## 124      6.3      2.7
## 125      6.7      3.3
## 126      7.2      3.2
## 127      6.2      2.8
## 128      6.1      3.0
## 129      6.4      2.8
## 130      7.2      3.0
## 131      7.4      2.8
## 132      7.9      3.8
## 133      6.4      2.8
## 134      6.3      2.8
## 135      6.1      2.6
## 136      7.7      3.0
## 137      6.3      3.4
## 138      6.4      3.1
## 139      6.0      3.0
## 140      6.9      3.1
## 141      6.7      3.1
## 142      6.9      3.1
## 143      5.8      2.7
## 144      6.8      3.2
## 145      6.7      3.3
## 146      6.7      3.0
## 147      6.3      2.5
## 148      6.5      3.0
## 149      6.2      3.4
## 150      5.9      3.0
```

- `starts_with()`: Starts with a prefix.
- `ends_with()`: Ends with a suffix.
- `contains()`: Contains a literal string.
- `matches()`: Matches a regular expression.
- `num_range()`: Matches a numerical range like 1:100.

rename 更改欄位名稱

```
rename(flights, ActualDepatureTime = dep_time)
```

```
## # A tibble: 336,776 × 19
##   year month   day ActualDepatureTime sched_dep_time dep_delay arr
##   <int> <int> <int>          <int>          <int>      <dbl>
##   <int>
## 1  2013     1     1             517            515         2
##   830
## 2  2013     1     1             533            529         4
##   850
```



```
## 3 2013 1 1 542 540 2
923
## 4 2013 1 1 544 545 -1
1004
## 5 2013 1 1 554 600 -6
812
## 6 2013 1 1 554 558 -4
740
## 7 2013 1 1 555 600 -5
913
## 8 2013 1 1 557 600 -3
709
## 9 2013 1 1 557 600 -3
838
## 10 2013 1 1 558 600 -2
753
## # i 336,766 more rows
## # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier
r <chr>,
## # flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time
<dbl>,
## # distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

distinct 選取不重複資料

```
set.seed(168)

df <- data.frame(
  x = sample(10, 100, replace = TRUE), # rep = replace
  y = sample(10, 100, rep = TRUE)
) # 100*2

head(df)

## x y
## 1 2 8
## 2 3 8
## 3 7 6
## 4 5 8
## 5 6 2
## 6 7 2

# 不重複列
distinct(df)

## x y
## 1 2 8
## 2 3 8
## 3 7 6
## 4 5 8
## 5 6 2
```

```
## 6 7 2
## 7 8 10
## 8 4 4
## 9 6 1
## 10 5 6
## 11 10 3
## 12 1 7
## 13 10 8
## 14 1 3
## 15 9 6
## 16 8 9
## 17 1 8
## 18 9 1
## 19 4 1
## 20 7 1
## 21 2 7
## 22 10 6
## 23 6 6
## 24 5 3
## 25 10 1
## 26 6 5
## 27 7 3
## 28 10 9
## 29 10 10
## 30 8 8
## 31 8 6
## 32 8 2
## 33 5 9
## 34 2 3
## 35 3 10
## 36 6 9
## 37 6 4
## 38 9 8
## 39 7 4
## 40 3 7
## 41 9 9
## 42 4 10
## 43 10 7
## 44 2 2
## 45 8 4
## 46 10 5
## 47 4 7
## 48 3 9
## 49 4 5
## 50 9 5
## 51 4 3
## 52 5 10
## 53 2 6
## 54 10 2
## 55 5 2
```

```

## 56 4 8
## 57 1 10
## 58 9 10
## 59 5 5
## 60 3 3
## 61 9 3
## 62 3 6
## 63 7 7

# nrow 列數
nrow(distinct(df))

## [1] 63

nrow(distinct(df, x, y))

## [1] 63

distinct(df, x) # 與下列結果相同 unique(df$x)

##      x
## 1    2
## 2    3
## 3    7
## 4    5
## 5    6
## 6    8
## 7    4
## 8   10
## 9    1
## 10   9

distinct(df, y) # 與下列結果相同 unique(df$y)

##      y
## 1    8
## 2    6
## 3    2
## 4   10
## 5    4
## 6    1
## 7    3
## 8    7
## 9    9
## 10   5

```

mutate 新增欄位

```

# mutate 新增欄位, 保留原資料
print(mutate(mtcars, displ_1 = disp / 61.0237))

```

## arb	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	c
## Mazda RX4 4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	
## Mazda RX4 Wag 4	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	
## Datsun 710 1	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	
## Hornet 4 Drive 1	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	
## Hornet Sportabout 2	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	
## Valiant 1	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	
## Duster 360 4	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	
## Merc 240D 2	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	
## Merc 230 2	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	
## Merc 280 4	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	
## Merc 280C 4	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	
## Merc 450SE 3	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	
## Merc 450SL 3	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	
## Merc 450SLC 3	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	
## Cadillac Fleetwood 4	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	
## Lincoln Continental 4	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	
## Chrysler Imperial 4	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	
## Fiat 128 1	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	
## Honda Civic 2	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	
## Toyota Corolla 1	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	
## Toyota Corona 1	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	
## Dodge Challenger 2	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	
## AMC Javelin 2	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	
## Camaro Z28 4	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	

## Pontiac Firebird 2	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3
## Fiat X1-9 1	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4
## Porsche 914-2 2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5
## Lotus Europa 2	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5
## Ford Pantera L 4	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5
## Ferrari Dino 6	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5
## Maserati Bora 8	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5
## Volvo 142E 2	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4
##										
## Mazda RX4	2.621932									
## Mazda RX4 Wag	2.621932									
## Datsun 710	1.769804									
## Hornet 4 Drive	4.227866									
## Hornet Sportabout	5.899347									
## Valiant	3.687092									
## Duster 360	5.899347									
## Merc 240D	2.403984									
## Merc 230	2.307300									
## Merc 280	2.746474									
## Merc 280C	2.746474									
## Merc 450SE	4.519556									
## Merc 450SL	4.519556									
## Merc 450SLC	4.519556									
## Cadillac Fleetwood	7.734700									
## Lincoln Continental	7.538055									
## Chrysler Imperial	7.210313									
## Fiat 128	1.289663									
## Honda Civic	1.240502									
## Toyota Corolla	1.165121									
## Toyota Corona	1.968088									
## Dodge Challenger	5.211090									
## AMC Javelin	4.981671									
## Camaro Z28	5.735477									
## Pontiac Firebird	6.554830									
## Fiat X1-9	1.294579									
## Porsche 914-2	1.971365									
## Lotus Europa	1.558411									
## Ford Pantera L	5.751864									
## Ferrari Dino	2.376126									
## Maserati Bora	4.932510									
## Volvo 142E	1.982836									

summarise - 計算加總

```
# 計算航班總數
flights %>%
  summarise(
    total_flights = n() # n() : 資料筆數
  )

## # A tibble: 1 × 1
##   total_flights
##         <int>
## 1         336776
```

summarise - 平均起飛延誤

```
flights %>%
  summarise(
    mean_dep_delay = mean(dep_delay, na.rm = TRUE)
  )

## # A tibble: 1 × 1
##   mean_dep_delay
##         <dbl>
## 1          12.6
```

summarise - 多個統計量

```
flights %>%
  summarise(
    avg = mean(dep_delay, na.rm = TRUE),
    sd = sd(dep_delay, na.rm = TRUE),
    min = min(dep_delay, na.rm = TRUE),
    max = max(dep_delay, na.rm = TRUE)
  )

## # A tibble: 1 × 4
##   avg    sd  min  max
##   <dbl> <dbl> <dbl> <dbl>
## 1  12.6  40.2  -43  1301
```

group_by + summarise

```
# 每家航空公司的平均延誤
flights %>%
  group_by(carrier) %>%
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE)
  )

## # A tibble: 16 × 2
##   carrier avg_delay
##   <chr>    <dbl>
```

```
## 1 9E      16.7
## 2 AA      8.59
## 3 AS      5.80
## 4 B6     13.0
## 5 DL      9.26
## 6 EV     20.0
## 7 F9     20.2
## 8 FL     18.7
## 9 HA      4.90
## 10 MQ     10.6
## 11 OO     12.6
## 12 UA     12.1
## 13 US      3.78
## 14 VX     12.9
## 15 WN     17.7
## 16 YV     19.0
```

```
# 計算航空公司的班次總計
flights %>%
  group_by(carrier) %>%
  summarise(
    flights = n()
  )
```

```
## # A tibble: 16 × 2
##   carrier flights
##   <chr>      <int>
## 1 9E      18460
## 2 AA     32729
## 3 AS       714
## 4 B6     54635
## 5 DL     48110
## 6 EV     54173
## 7 F9       685
## 8 FL     3260
## 9 HA       342
## 10 MQ     26397
## 11 OO        32
## 12 UA     58665
## 13 US     20536
## 14 VX      5162
## 15 WN     12275
## 16 YV       601
```

group_by 多群組 + summarise

```
# 依航空公司與月份
flights %>%
  group_by(carrier, month) %>%
  summarise(
```

```

    avg_delay = mean(dep_delay, na.rm = TRUE)
  )
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by carrier and month.
## i Output is grouped by carrier.
## i Use `summarise(groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(carrier, month))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.

## # A tibble: 185 × 3
## # Groups:   carrier [16]
##   carrier month avg_delay
##   <chr>   <int>   <dbl>
## 1 9E       1      16.9
## 2 9E       2      16.5
## 3 9E       3      13.4
## 4 9E       4      13.6
## 5 9E       5      22.7
## 6 9E       6      29.0
## 7 9E       7      31.4
## 8 9E       8      17.3
## 9 9E       9       7.75
## 10 9E      10       9.33
## # i 175 more rows

```

計算航空公司的平均延誤時間, 由大至小排序

```

flights %>%
  group_by(carrier) %>%
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE)
  ) %>%
  arrange(desc(avg_delay))

## # A tibble: 16 × 2
##   carrier avg_delay
##   <chr>   <dbl>
## 1 F9      20.2
## 2 EV      20.0
## 3 YV      19.0
## 4 FL      18.7
## 5 WN      17.7
## 6 9E      16.7
## 7 B6      13.0
## 8 VX      12.9
## 9 OO      12.6
## 10 UA      12.1
## 11 MQ      10.6
## 12 DL       9.26

```



```
## 13 AA      8.59
## 14 AS      5.80
## 15 HA      4.90
## 16 US      3.78
```

dplyr in R Cheat Sheet

<https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf>

熟悉開放資料集於 dplyr 應用

- 2025.07.12_DATASETS_WISHLIST
 - https://github.com/rwepa/r_data_scientist/tree/main/r_training_advanced_ppt_codes_2025#20250712_datasets_wishlist

學員練習

🌸 2.3 tidyr : pivot_longer/pivot_wider、分欄/合欄

長寬資料簡介

- tidyr 套件中的 pivot_longer() (將寬表格轉為長表格) 與 pivot_wider() (將長表格轉為寬表格) 是 R 語言中重塑資料 (Reshaping Data) 的核心函數。
- pivot_longer 寬轉長: 當你的資料中欄位名稱包含資料變數 (例如把年份 2021, 2022 放成欄位名) 時, 非常適合使用 pivot_longer.
- pivot_longer 範例

```
library(tidyr)

##
## Attaching package: 'tidyr'

## The following object is masked from 'package:magrittr':
##
##      extract

library(dplyr)

# 建立寬表格 (Wide Format)
df_wide <- tibble(
  student = c("Alice", "Bob"),
  Math = c(90, 80),
  English = c(85, 95)
)

print(df_wide)
```

```
## # A tibble: 2 × 3
##   student Math English
##   <chr>   <dbl>   <dbl>
## 1 Alice     90     85
## 2 Bob      80     95

# 轉為長表格 (Long Format)
df_long <- df_wide %>%
  pivot_longer(
    cols = c(Math, English), # 要合併的欄位 (也可以寫 -student)
    names_to = "subject",    # 舊欄位名稱要放入的新欄位名稱
    values_to = "score"      # 舊欄位數值要放入的新欄位名稱
  )

print(df_long) # subject 欄位已經改為長資料

## # A tibble: 4 × 3
##   student subject score
##   <chr>   <chr>   <dbl>
## 1 Alice   Math     90
## 2 Alice   English  85
## 3 Bob     Math     80
## 4 Bob     English  95
```

- pivot_wider 長轉寬: 當你需要將資料做交叉表統計, 或是有些欄位的類別需要拆分成獨立欄位時, 使用 pivot_wider.
- pivot_wider 範例

```
# 轉為寬表格 (Wide Format)
df_recreated_wide <- df_long %>%
  pivot_wider(
    names_from = subject, # 哪一個欄位的值做為新欄位名稱
    values_from = score   # 哪一個欄位的值填入新欄位中
  )

print(df_recreated_wide)

## # A tibble: 2 × 3
##   student Math English
##   <chr>   <dbl>   <dbl>
## 1 Alice     90     85
## 2 Bob      80     95
```

- 寬表格的欄位名包含多個資訊 (例如 Math_2021, Math_2022)

```
# 包含科目與年份的複合欄位名
df_complex <- tibble(
  student = c("Alice", "Bob"),
  Math_2021 = c(90, 80),
```

```

Math_2022 = c(92, 85),
English_2021 = c(85, 95),
English_2022 = c(88, 98)
)
print(df_complex)

## # A tibble: 2 × 5
##   student Math_2021 Math_2022 English_2021 English_2022
##   <chr>      <dbl>    <dbl>      <dbl>      <dbl>
## 1 Alice         90        92         85         88
## 2 Bob           80        85         95         98

# 使用 names_sep 或 names_pattern 一次拆分成兩個新欄位
df_complex_long <- df_complex %>%
  pivot_longer(
    cols = -student, # 負號表示除了此欄位的其他所有欄位
    names_to = c("subject", "year"), # 長資料的 2 個新欄位名稱
    names_sep = "_", # 根據底線拆分
    values_to = "score" # 長資料的值
  )

print(df_complex_long)

## # A tibble: 8 × 4
##   student subject year  score
##   <chr>   <chr>   <chr> <dbl>
## 1 Alice   Math    2021    90
## 2 Alice   Math    2022    92
## 3 Alice   English 2021    85
## 4 Alice   English 2022    88
## 5 Bob     Math    2021    80
## 6 Bob     Math    2022    85
## 7 Bob     English 2021    95
## 8 Bob     English 2022    98

```

tidyr + ggplot2 → 使用 tidyverse 整合套件

1. 載入所需套件

```
library(tidyverse) # 包含 tidyr, ggplot2, dplyr, readr 等
```

```

## — Attaching core tidyverse packages ————— tidyverse
## rse 2.0.0 —
## ✓ forcats    1.0.1    ✓ purrr      1.2.2
## ✓ ggplot2    4.0.3    ✓ readr      2.2.0
## ✓ lubridate  1.9.5    ✓ stringr    1.6.0
## — Conflicts ————— tidyverse_conflicts() —
## ✗ tidyr::extract() masks magrittr::extract()
## ✗ dplyr::filter()  masks stats::filter()

```

```
## X dplyr::lag()          masks stats::lag()
## X purrr::set_names() masks magrittr::set_names()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

2. 建立模擬資料 (寬表格: 包含 Month, Product_A, Product_B, Product_C)

```
set.seed(42)
df_sales_wide <- tibble(
  month = 1:12,
  Product_A = cumsum(rnorm(12, mean = 5, sd = 2)) + 100,
  Product_B = cumsum(rnorm(12, mean = 3, sd = 3)) + 80,
  Product_C = cumsum(rnorm(12, mean = 7, sd = 1.5)) + 60
)
```

```
print("原始寬表格 (df_sales_wide) : ")
```

```
## [1] "原始寬表格 (df_sales_wide) : "
```

```
print(head(df_sales_wide, 3))
```

```
## # A tibble: 3 x 4
##   month Product_A Product_B Product_C
##   <int>     <dbl>     <dbl>     <dbl>
## 1     1      108.       78.8       69.8
## 2     2      112.       81.0       76.2
## 3     3      117.       83.6       82.8
```

3. 使用 pivot_longer 轉為長表格

```
df_sales_long <- df_sales_wide %>%
  pivot_longer(
    cols = starts_with("Product_"), # 選擇要合併的欄位 (也可以寫 -month)
    names_to = "product_type",      # 舊欄位名稱轉入此新欄位
    values_to = "sales_amount"     # 舊欄位數值轉入此新欄位
  )
```

```
print("轉換後的長表格 (df_sales_long) : ")
```

```
## [1] "轉換後的長表格 (df_sales_long) : "
```

```
print(head(df_sales_long, 6))
```

```
## # A tibble: 6 x 3
##   month product_type sales_amount
##   <int> <chr>          <dbl>
## 1     1 Product_A      108.
## 2     1 Product_B      78.8
## 3     1 Product_C      69.8
## 4     2 Product_A      112.
## 5     2 Product_B      81.0
## 6     2 Product_C      76.2
```

4. 結合 *ggplot2* 繪製多指標折線圖

```
ggplot(df_sales_long, aes(x = month, y = sales_amount, color = product_
type)) +
```

```
# 繪製折線與資料點
```

```
geom_line(linewidth = 1) +
geom_point(size = 2.5) +
```

```
# 設定 X 軸刻度 (顯示 1~12 月)
```

```
scale_x_continuous(breaks = 1:12) +
```

```
# 設定顏色與圖例名稱
```

```
scale_color_brewer(
  palette = "Set1",
  name = "產品類別",
  labels = c("產品 A", "產品 B", "產品 C")
) +
```

```
# 標題與軸標籤
```

```
labs(
  title = "2025 年各產品月銷售額趨勢圖",
  subtitle = "使用 tidyr::pivot_longer 轉檔後繪製",
  x = "月份",
  y = "銷售額 (萬元)",
  caption = "資料來源：00 企業銷售部"
) +
```

```
# 套用簡潔主題與細節微調
```

```
theme_minimal(base_size = 12) + # 簡潔主題風格(無背景填色，無外框線，保
留網格線)，基礎文字大小=12
```

```
theme(
```

```
# 將標題置中對齊 (hjust = 0.5)，並加粗
```

```
plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
```

```
# 副標題一起置中對齊
```

```
plot.subtitle = element_text(hjust = 0.5, size = 12, color = "gray3
0"),
```

```
# 將圖例移至下方
```

```
legend.position = "bottom",
```

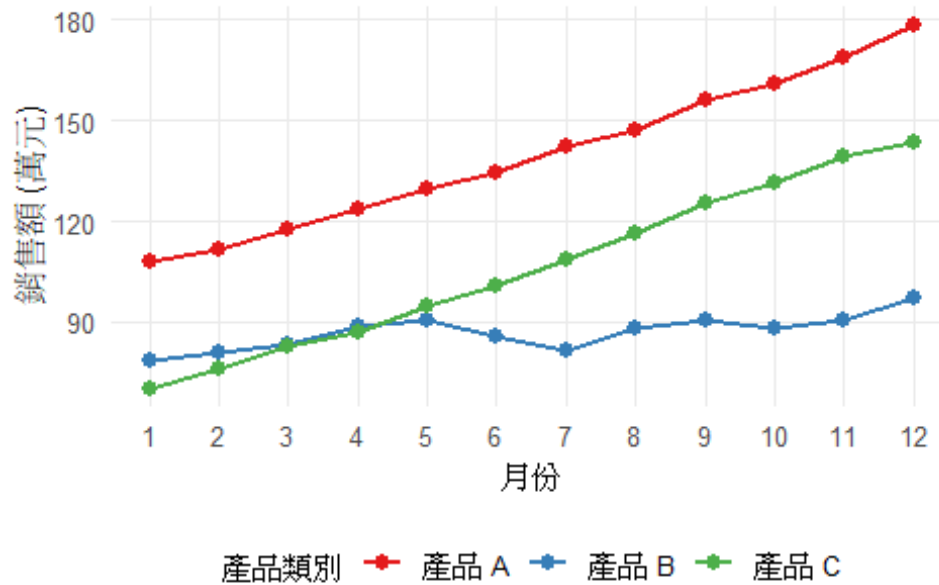
```
# 移除次要網格線
```

```
panel.grid.minor = element_blank()
```

```
)
```

2025 年各產品月銷售額趨勢圖

使用 tidyr::pivot_longer 轉檔後繪製



資料來源：OO企業銷售部

近十年國防預算結構分析

- 下載: <https://data.gov.tw/dataset/139911>
- 檔名: 近十年國防預算結構分析_290973.csv

載入套件

```
library(readr)
```

```
library(dplyr)
```

匯入 CSV

```
df <- read_csv("近十年國防預算結構分析_290973.csv")
```

```
## Rows: 46 Columns: 8
```

```
## — Column specification —————
```

```
## Delimiter: ","
```

```
## chr (4): ItemName, Category, Author, URL
```

```
## dbl (4): SeqNo, Annual, Percentage, IssueDate
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

顯示資料集

```
df # 46*8
```

```
## # A tibble: 46 × 8
##   SeqNo Annual ItemName          Category Percentage Author Iss
ueDate URL
##   <dbl> <dbl> <chr>          <chr>          <dbl> <chr>
<dbl> <chr>
## 1      1      101 近十年國防預算結構分析…… 軍事投資      27.3 國防部主
計… 20210521 http…
## 2      2      101 近十年國防預算結構分析…… 作業維持      22.3 國防部主
計… 20210521 http…
## 3      3      101 近十年國防預算結構分析…… 人員維持      49    國防部主
計… 20210521 http…
## 4      4      101 近十年國防預算結構分析…… 其他          1.41 國防部主
計… 20210521 http…
## 5      5      102 近十年國防預算結構分析…… 軍事投資      26.0 國防部主
計… 20210521 http…
## 6      6      102 近十年國防預算結構分析…… 作業維持      22.6 國防部主
計… 20210521 http…
## 7      7      102 近十年國防預算結構分析…… 人員維持      49.9 國防部主
計… 20210521 http…
## 8      8      102 近十年國防預算結構分析…… 其他          1.48 國防部主
計… 20210521 http…
## 9      9      103 近十年國防預算結構分析…… 軍事投資      27.6 國防部主
計… 20210521 http…
## 10     10     103 近十年國防預算結構分析…… 作業維持      22.0 國防部主
計… 20210521 http…
## # i 36 more rows
```

- 資料檢視函數,快速查看資料框 (data.frame 或 tibble) 的結構.

```
glimpse(df)

## Rows: 46
## Columns: 8
## $ SeqNo      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
16, 17, ...
## $ Annual     <dbl> 101, 101, 101, 101, 102, 102, 102, 102, 103, 103,
103, 103,...
## $ ItemName   <chr> "近十年國防預算結構分析", "近十年國防預算結構分析", "
近十年國防預算結構分析", "近十年國防預算結構分析",...
## $ Category   <chr> "軍事投資", "作業維持", "人員維持", "其他", "軍事投
資", "作業維持", "人員維持", "其他",...
## $ Percentage <dbl> 27.26, 22.33, 49.00, 1.41, 25.98, 22.62, 49.92, 1.
48, 27.62...
## $ Author     <chr> "國防部主計局", "國防部主計局", "國防部主計局", "國防
部主計局", "國防部主計局", "國防部主計局",...
## $ IssueDate  <dbl> 20210521, 20210521, 20210521, 20210521, 20210521,
20210521,...
```

```
## $ URL      <chr> "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d...
```

- 資料結構

```
str(df)
```

```
## spc_tbl_ [46 × 8] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ SeqNo      : num [1:46] 1 2 3 4 5 6 7 8 9 10 ...
## $ Annual     : num [1:46] 101 101 101 101 102 102 102 102 103 103 ...
## $ ItemName    : chr [1:46] "近十年國防預算結構分析" "近十年國防預算結構分析" "近十年國防預算結構分析" "近十年國防預算結構分析" ...
## $ Category   : chr [1:46] "軍事投資" "作業維持" "人員維持" "其他" ...
## $ Percentage : num [1:46] 27.26 22.33 49 1.41 25.98 ...
## $ Author     : chr [1:46] "國防部主計局" "國防部主計局" "國防部主計局" "國防部主計局" ...
## $ IssueDate  : num [1:46] 20210521 20210521 20210521 20210521 20210521 ...
## $ URL        : chr [1:46] "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d%81%e5%b9%b4%e5%9c%8b%e9%98%b2%e9%a0%90%e7%ae%97%e7%b" | __truncated__ "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d%81%e5%b9%b4%e5%9c%8b%e9%98%b2%e9%a0%90%e7%ae%97%e7%b" | __truncated__ "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d%81%e5%b9%b4%e5%9c%8b%e9%98%b2%e9%a0%90%e7%ae%97%e7%b" | __truncated__ "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d%81%e5%b9%b4%e5%9c%8b%e9%98%b2%e9%a0%90%e7%ae%97%e7%b" | __truncated__ "https://www.mnd.gov.tw/NewUpload/202105/12.%e8%bf%91%e5%8d%81%e5%b9%b4%e5%9c%8b%e9%98%b2%e9%a0%90%e7%ae%97%e7%b" | __truncated__ ...
## - attr(*, "spec")=
## .. cols(
## ..   SeqNo = col_double(),
## ..   Annual = col_double(),
## ..   ItemName = col_character(),
## ..   Category = col_character(),
## ..   Percentage = col_double(),
## ..   Author = col_character(),
## ..   IssueDate = col_double(),
## ..   URL = col_character()
## .. )
## - attr(*, "problems")=<pointer: 0x000001ac975ff800>
```

- 資料摘要

```
summary(df) # 有NA
```

```
##      SeqNo      Annual      ItemName      Category      Per
centage
## Min.   : 1.00   Min.   :101.0   Length   :46   Length   :46   Min.
: 1.41
## 1st Qu.:12.25   1st Qu.:103.2   N.unique : 1   N.unique : 4   1st Q
u.:22.95
## Median :23.50   Median :106.0   N.blank  : 0   N.blank  : 0   Media
n :27.24
```



```
## Mean :23.50 Mean :106.5 Min.nchar:11 Min.nchar: 2 Mean
:28.26
## 3rd Qu.:34.75 3rd Qu.:109.8 Max.nchar:11 Max.nchar: 4 3rd Q
u.:43.21
## Max. :46.00 Max. :113.0 Max.
:49.92
## Author IssueDate URL
## Length :46 Min. :20210521 Length : 46
## N.unique : 1 1st Qu.:20210521 N.unique : 4
## N.blank : 0 Median :20210521 N.blank : 0
## Min.nchar: 6 Mean :20215100 Min.nchar:153
## Max.nchar: 6 3rd Qu.:20210521 Max.nchar:181
## Max. :20240531
```

前6筆

head(df)

```
## # A tibble: 6 × 8
## SeqNo Annual ItemName Category Percentage Author Iss
ueDate URL
## <dbl> <dbl> <chr> <chr> <dbl> <chr>
<dbl> <chr>
## 1 1 101 近十年國防預算結構分析 軍事投資 27.3 國防部主計…
20210521 http…
## 2 2 101 近十年國防預算結構分析 作業維持 22.3 國防部主計…
20210521 http…
## 3 3 101 近十年國防預算結構分析 人員維持 49 國防部主計…
20210521 http…
## 4 4 101 近十年國防預算結構分析 其他 1.41 國防部主計…
20210521 http…
## 5 5 102 近十年國防預算結構分析 軍事投資 26.0 國防部主計…
20210521 http…
## 6 6 102 近十年國防預算結構分析 作業維持 22.6 國防部主計…
20210521 http…
```

後6筆

tail(df)

```
## # A tibble: 6 × 8
## SeqNo Annual ItemName Category Percentage Author Iss
ueDate URL
## <dbl> <dbl> <chr> <chr> <dbl> <chr>
<dbl> <chr>
## 1 41 112 近十年國防預算結構分析 軍事投資 23.7 國防部主計…
20240531 http…
## 2 42 112 近十年國防預算結構分析 作業維持 32.6 國防部主計…
20240531 http…
## 3 43 112 近十年國防預算結構分析 人員維持 43.7 國防部主計…
20240531 http…
```

## 4	44	113	近十年國防預算結構分析 軍事投資	28.4	國防部主計...
20240523	http...				
## 5	45	113	近十年國防預算結構分析 作業維持	29.7	國防部主計...
20240523	http...				
## 6	46	113	近十年國防預算結構分析 人員維持	41.9	國防部主計...
20240523	http...				

長資料(1 行軍事投資,作業維持,人員維持,其他)轉換為寬資料(4 行)

```
library(tidyr) # spread
library(dplyr)
```

方法 1 不使用管線操作

```
defense_long <- select(df, Annual:Percentage)

defense_long <- select(defense_long, -c(ItemName))

# spread 表示將長資料轉換為寬資料
defense_wide <- spread(defense_long, key=Category, value=Percentage)
```

defense_wide

```
## # A tibble: 13 x 5
##   Annual 人員維持 作業維持 其他 軍事投資
##   <dbl>   <dbl>   <dbl> <dbl>   <dbl>
## 1    101     49    22.3  1.41    27.3
## 2    102    49.9    22.6  1.48    26.0
## 3    103    48.9    22.0  1.45    27.6
## 4    104    45.2    22.5  1.55    30.8
## 5    105    45.5    22.7  1.44    30.3
## 6    106    46.3    25.4  1.48    26.8
## 7    107    48.4    25.4  1.46    24.8
## 8    108    46.0    27.7  NA      26.4
## 9    109    47.3    27.2  NA      25.5
## 10   110    46.9    27.4  NA      25.7
## 11   111    47.4    28.5  NA      24.1
## 12   112    43.7    32.6  NA      23.7
## 13   113    41.9    29.7  NA      28.4
```

方法 2 使用 %>%

```
defense_wide <- df %>%
  select(Annual:Percentage) %>%
  select(-c(ItemName)) %>%
  spread(key=Category, value=Percentage)
```

defense_wide

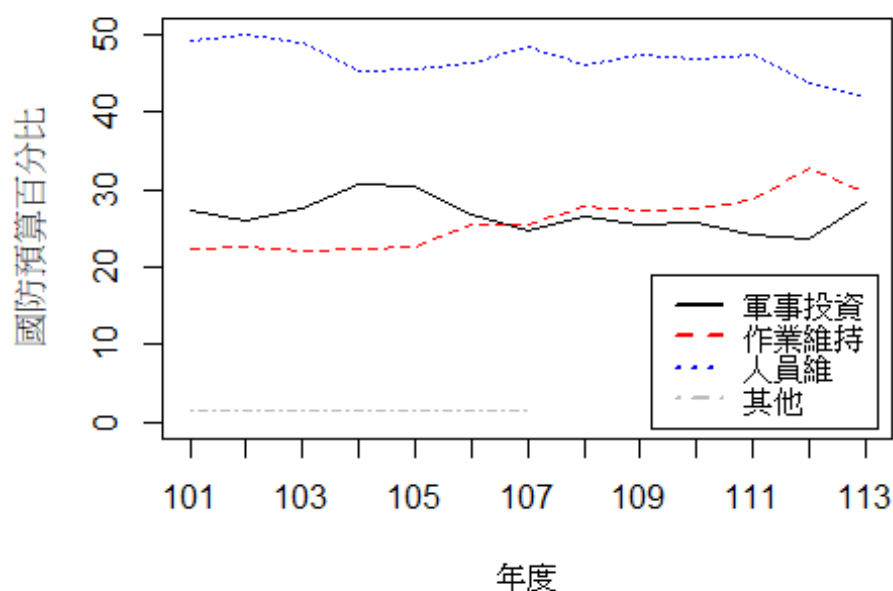
```
## # A tibble: 13 × 5
##   Annual 人員維持 作業維持 其他 軍事投資
##   <dbl>   <dbl>   <dbl> <dbl>   <dbl>
## 1    101     49    22.3  1.41    27.3
## 2    102    49.9    22.6  1.48    26.0
## 3    103    48.9    22.0  1.45    27.6
## 4    104    45.2    22.5  1.55    30.8
## 5    105    45.5    22.7  1.44    30.3
## 6    106    46.3    25.4  1.48    26.8
## 7    107    48.4    25.4  1.46    24.8
## 8    108    46.0    27.7 NA        26.4
## 9    109    47.3    27.2 NA        25.5
## 10   110    46.9    27.4 NA        25.7
## 11   111    47.4    28.5 NA        24.1
## 12   112    43.7    32.6 NA        23.7
## 13   113    41.9    29.7 NA        28.4
```

國防預算結構統計圖

```
plot(defense_wide$軍事投資, type="l",
     xlab="年度",
     ylab="國防預算百分比",
     main="101~113 年國防預算結構統計圖",
     axes=FALSE,
     cex=0.5,
     ylim=c(0, max(defense_wide[2:5], na.rm=TRUE)))
lines(defense_wide$作業維持, col="red", lty=2)
lines(defense_wide$人員維持, col="blue", lty=3)
lines(defense_wide$其他, col="grey", lty=4)

#ind <- round(seq(1, nrow(defense_wide), length.out=5))
axis(1, at=c(1:13), labels=defense_wide$Annual)
axis(2)
legend(x="bottomright",
       inset = c(0.02, 0.02),
       legend=c("軍事投資", "作業維持", "人員維", "其他"),
       col=c("black", "red", "blue", "grey"),
       lty=c(1:4),
       lwd=2,
       cex = 0.8)
box()
```

101~113年國防預算結構統計圖



```
# legend(x 設定值= "xxx")
# topleft    top    topright
# left      center right
# bottomleft bottom bottomright
```

- 練習使用 `pivot_wider`, `pivot_longer` {tidyr} 進行長寬資料轉換

🌸 2.4 合併資料：join 家族與鍵值設計

合併簡介

- `tidyverse` 中的 `join` 是資料整合 (Data Joining) 的核心功能，主要由 `dplyr` 套件提供。它可以依據一個或多個共同欄位 (Key)，將兩個資料框 (`data.frame` 或 `tibble`) 合併，功能類似 SQL 的 JOIN。
- Join 的目的通常是「將右表 (第二個資料框) 的欄位加入左表 (第一個資料框)」。
- 共同欄位 (ID) 只是拿來配對，一般不會新增右表之共同欄位。
- `dplyr` 套件六種合併資料

函數	保留左表	保留右表	新增右表欄位	常見用途
----	------	------	--------	------

函數	保留左表	保留右表	新增右表欄位	常見用途
inner_join()	共同資料	共同資料	✓	交集分析
left_join()	✓	只有符合者	✓	最常用，補充欄位
right_join()	只有符合者	✓	✓	保留右表
full_join()	✓	✓	✓	完整整合資料
semi_join()	✓ (符合者)	✗	✗	篩選存在於右表的資料
anti_join()	✓ (未符合者)	✗	✗	找遺漏、異常資料

- 註 1: semi_join() 的用途不是合併資料，而是依據右表是否存在符合條件的鍵值來篩選左表資料，因此只回傳左表原本的欄位。
- 註 2: anti_join() 的目的也是篩選，只留下左表中找不到配對的列，因此同樣不會加入右表的任何欄位。

範例 1 left_join - 左表新增右表的 1 個欄位

```
library(dplyr)

students <- tibble(
  ID = c(1,2,3),
  Name = c("Alan", "Bob", "Cathy")
)
print("student - 左表")

## [1] "student - 左表"

print(students)

## # A tibble: 3 × 2
##       ID Name
##   <dbl> <chr>
## 1     1 Alan
## 2     2 Bob
## 3     3 Cathy

scores <- tibble(
  ID = c(1,2,3),
  Math = c(90,80,95)
)
print("scores - 右表")

## [1] "scores - 右表"

print(scores)
```

```
## # A tibble: 3 × 2
##       ID  Math
##   <dbl> <dbl>
## 1     1    90
## 2     2    80
## 3     3    95

students %>%
  left_join(scores, by = "ID")

## # A tibble: 3 × 3
##       ID Name  Math
##   <dbl> <chr> <dbl>
## 1     1  Alan    90
## 2     2  Bob     80
## 3     3  Cathy   95
```

範例 2 left_join - 左表新增右表的多個欄位

```
scores <- tibble(
  ID = c(1,2,3),
  Math = c(90,80,95),
  English = c(88,76,91),
  Science = c(92,81,94)
)

students %>%
  left_join(scores, by="ID")

## # A tibble: 3 × 5
##       ID Name  Math English Science
##   <dbl> <chr> <dbl>   <dbl>   <dbl>
## 1     1  Alan    90      88      92
## 2     2  Bob     80      76      81
## 3     3  Cathy   95      91      94
```

範例 3 left_join - 右表的欄位找不到

```
print(students)

## # A tibble: 3 × 2
##       ID Name
##   <dbl> <chr>
## 1     1  Alan
## 2     2  Bob
## 3     3  Cathy

scores <- tibble(
  ID = c(1,2),
  Math = c(90,80)
```

```

)
print("scores - 右表")

## [1] "scores - 右表"

print(scores)

## # A tibble: 2 × 2
##       ID  Math
##   <dbl> <dbl>
## 1     1    90
## 2     2    80

# 新增結果為 NA
students %>%
  left_join(scores)

## Joining with `by = join_by(ID)`

## # A tibble: 3 × 3
##       ID Name  Math
##   <dbl> <chr> <dbl>
## 1     1  Alan    90
## 2     2  Bob     80
## 3     3  Cathy   NA

```

🌸 2.5 練習：完成一份可分析的乾淨資料集

參考 [Open Data](#) 等資料，完成 Tidy Data。

熟悉以下資料集之資料操作

1. AirPassengers : ts 時間序列
2. Titanic : table 表格
3. airquality : data.frame 資料框
4. iris : data.frame 資料框
5. mtcars : data.frame 資料框
6. penguins : data.frame 資料框

1. AirPassengers 空中旅客人數資料集 —

- 1949-1960 年航空旅客每月平均乘客數,單位:千人.
- 12 年*12 個月=144 筆資料
- 時間序列是一組按照時間發生先後順序進行排列的資料序列.
- 通常一組時間序列的時間間隔為一固定值.

- https://github.com/rwepa/r_data_scientist/blob/main/r_training_ppt_codes_2025/r_data_science_02_r_object.R#L652

```
?AirPassengers
```

```
## starting httpd help server ... done
```

```
data("AirPassengers")
```

```
AirPassengers
```

```
##      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
## 1949 112 118 132 129 121 135 148 148 136 119 104 118
## 1950 115 126 141 135 125 149 170 170 158 133 114 140
## 1951 145 150 178 163 172 178 199 199 184 162 146 166
## 1952 171 180 193 181 183 218 230 242 209 191 172 194
## 1953 196 196 236 235 229 243 264 272 237 211 180 201
## 1954 204 188 235 227 234 264 302 293 259 229 203 229
## 1955 242 233 267 269 270 315 364 347 312 274 237 278
## 1956 284 277 317 313 318 374 413 405 355 306 271 306
## 1957 315 301 356 348 355 422 465 467 404 347 305 336
## 1958 340 318 362 348 363 435 491 505 404 359 310 337
## 1959 360 342 406 396 420 472 548 559 463 407 362 405
## 1960 417 391 419 461 472 535 622 606 508 461 390 432
```

```
class(AirPassengers) # ts
```

```
## [1] "ts"
```

```
length(AirPassengers)
```

```
## [1] 144
```

```
str(AirPassengers)
```

```
## Time-Series [1:144] from 1949 to 1961: 112 118 132 129 121 135 148
148 136 119 ...
```

```
summary(AirPassengers)
```

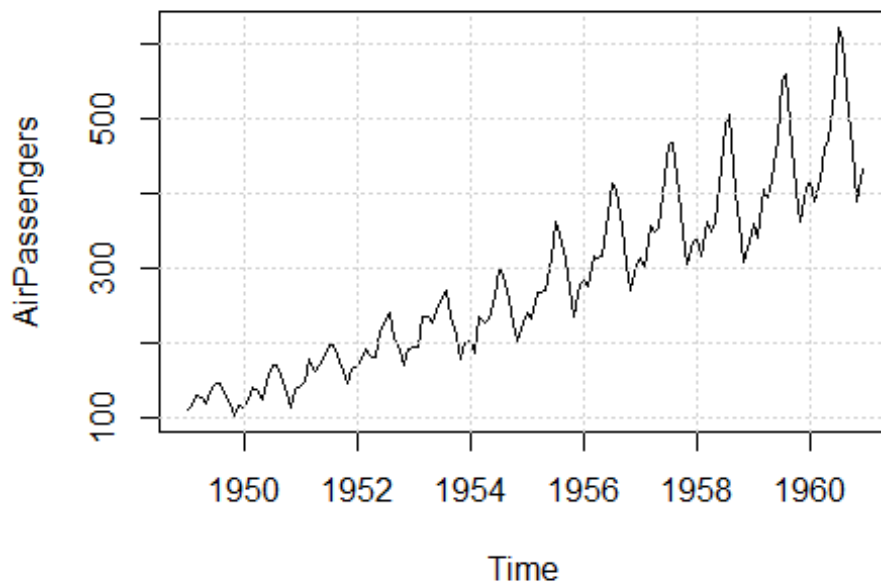
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  104.0   180.0   265.5   280.3   360.5   622.0
```

```
# 繪製 ts 資料
```

```
plot(AirPassengers, main="AirPassengers 時間序列圖")
```

```
grid()
```


AirPassengers時間序列圖



```
# 找出樣式 (Patterns)
```

2. Titanic 1912 年鐵達尼號資料集

```
?Titanic
```

```
data(Titanic)
```

Titanic # *Titanic* 不是 *data.frame*, 而是一個四維列聯表 (*Contingency Table*)

```
## , , Age = Child, Survived = No
```

```
##
```

```
##      Sex
```

```
## Class  Male  Female
```

```
## 1st      0      0
```

```
## 2nd      0      0
```

```
## 3rd     35     17
```

```
## Crew      0      0
```

```
##
```

```
## , , Age = Adult, Survived = No
```

```
##
```

```
##      Sex
```

```
## Class  Male  Female
```

```
## 1st    118      4
```

```
## 2nd    154     13
```

```
## 3rd    387     89
```

```
## Crew   670      3
```

```
##
## , , Age = Child, Survived = Yes
##
##      Sex
## Class  Male Female
## 1st      5      1
## 2nd     11     13
## 3rd     13     14
## Crew      0      0
##
## , , Age = Adult, Survived = Yes
##
##      Sex
## Class  Male Female
## 1st     57     140
## 2nd     14      80
## 3rd     75      76
## Crew   192      20

class(Titanic) # table

## [1] "table"

sum(Titanic) # 總人數

## [1] 2201

# 物件結構, 有4個維度(列: Class, 行: Sex, 第3維度: Age, 第4維度: Survived)
# 4*2*2*2 = 32 種組合
str(Titanic)

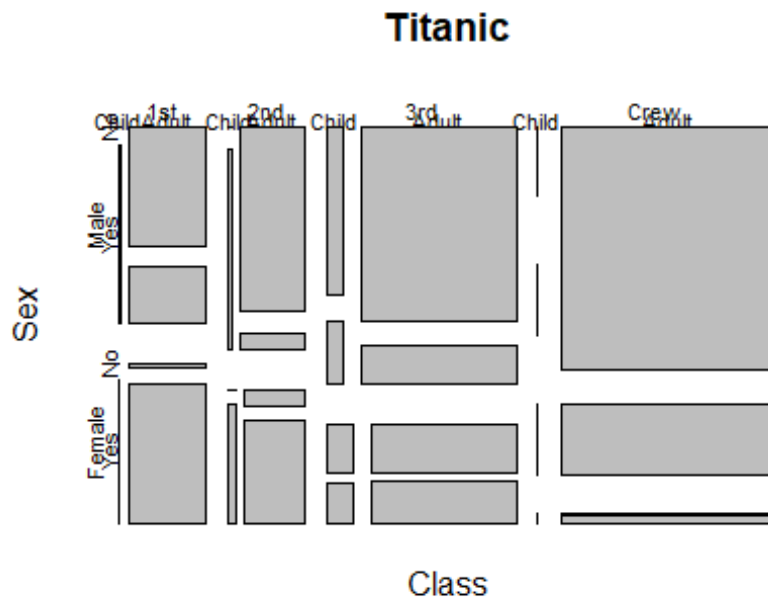
## 'table' num [1:4, 1:2, 1:2, 1:2] 0 0 35 0 0 0 17 0 118 154 ...
## - attr(*, "dimnames")=List of 4
## ..$ Class : chr [1:4] "1st" "2nd" "3rd" "Crew"
## ..$ Sex : chr [1:2] "Male" "Female"
## ..$ Age : chr [1:2] "Child" "Adult"
## ..$ Survived: chr [1:2] "No" "Yes"

summary(Titanic)

## Number of cases in table: 2201
## Number of factors: 4
## Test for independence of all factors:
## Chisq = 1637.4, df = 25, p-value = 0
## Chi-squared approximation may be incorrect

# 繪製 table 資料
# Mosaic Plot (馬賽克圖)是一種專門用來呈現多維類別資料 (Multivariate Categorical Data)的圖形.
# 馬賽克圖用來呈現 32 種組合的人數. 矩形越大, 代表該組合的人數越多.
```

```
# Crew > 3rd (三等艙) > 1st(頭等艙) > 2nd(二等艙)
plot(Titanic)
```



```
# 兒童存活率較高嗎？
apply(Titanic, c(3, 4), sum)
```

```
##           Survived
## Age          No Yes
## Child       52  57
## Adult     1438 654
```

```
# 女性的存活率更高？
apply(Titanic, c(2, 4), sum)
```

```
##           Survived
## Sex          No Yes
## Male       1364 367
## Female     126 344
```

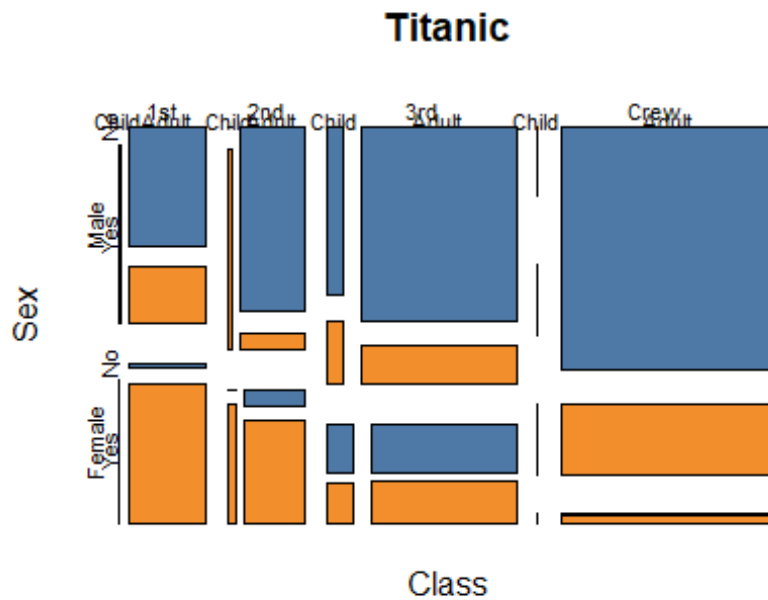
```
# 使用 mosaicplot() 自訂顏色
```

```
mosaicplot(
  Titanic,
  color = c(
    "#4E79A7", # 藍
    "#F28E2B", # 橘
    "#59A14F", # 綠
    "#E15759", # 紅
  )
)
```

```

"#76B7B2", # 青
"#EDC948", # 黄
"#B07AA1", # 紫
"#FF9DA7"  # 粉
),
main = "Titanic",
xlab = "Class",
ylab = "Sex"
)

```



```

# 依 Survival 著色

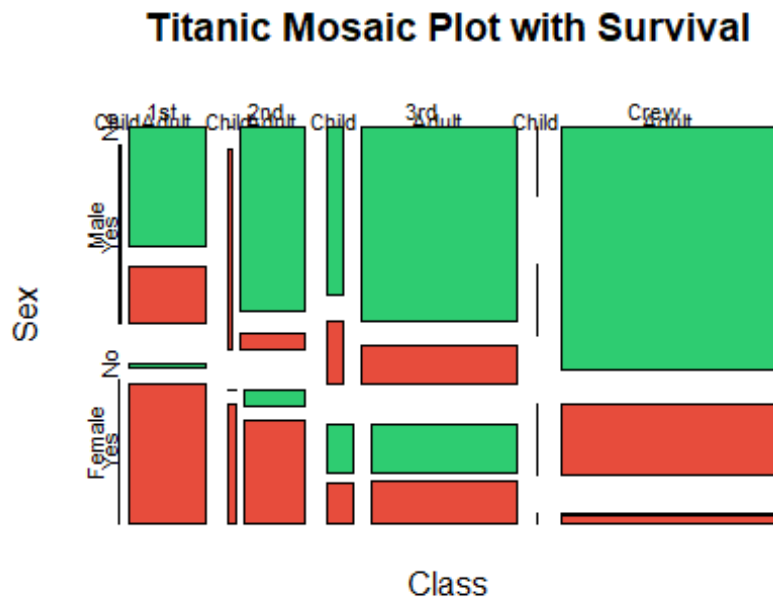
# 生還 (Yes): 綠色
# 死亡 (No) : 紅色

cols <- c(
  "#2ECC71", # 生還
  "#E74C3C"  # 死亡
)

mosaicplot(
  Titanic,
  color = cols,
  main = "Titanic Mosaic Plot with Survival",
  xlab = "Class",

```

```
ylab = "Sex"
)
```

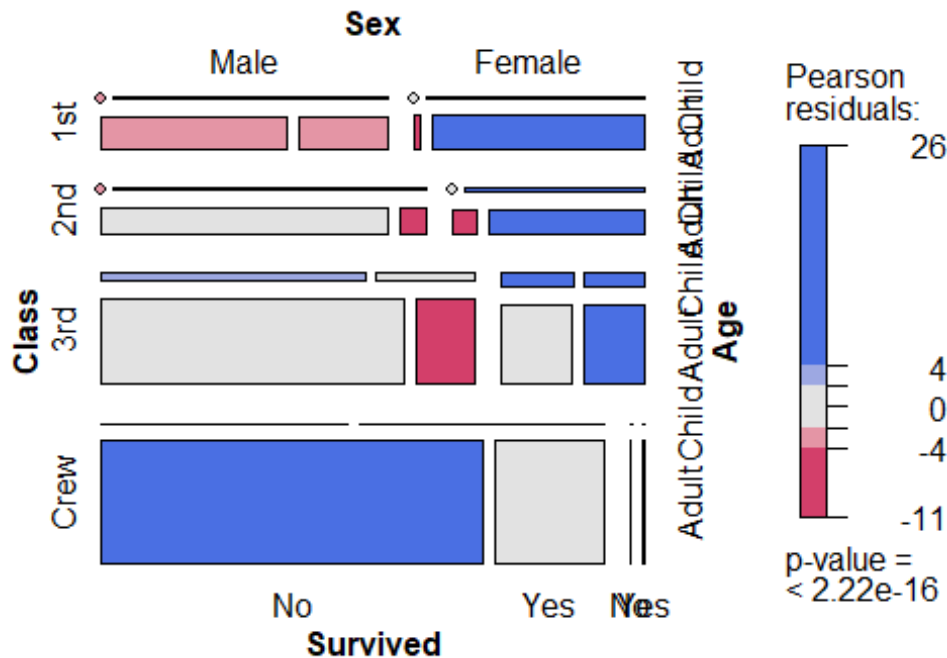


```
# 使用 vcd 套件
# install.packages("vcd")
library(vcd)

## Loading required package: grid

mosaic(
  Titanic,
  shade = TRUE,
  legend = TRUE,
  main = "Titanic Mosaic Plot with vcd package"
)
```

Titanic Mosaic Plot with vcd package

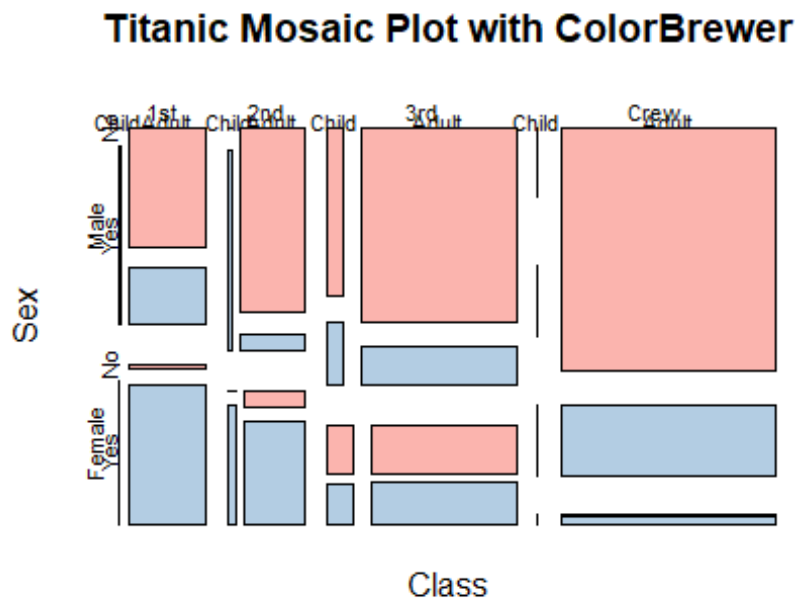


- # 左側 (Y 軸) Class, 依序為: 1st, 2nd, 3rd, Crew, 每一列代表一種艙等。
- # 上方 Sex, 區分成: Male, Female, 左右兩區比較男女差異。
- # 下方 Survived, 區分成: No, Yes, 可以直接比較 死亡 vs 生還。
- # 最右側 Age, 區分成: Adult, Child。
- # 右側圖例顏色代表 Pearson residuals, 即 Pearson 標準化殘差, 表示'實際人數'與'理論期望人數'的差異。
- # 藍色表示 實際人數 > 期望值人數, Observed > Expected. 例如: 頭等艙女性生還, Crew 男性死亡 都是深藍色。
- # 紅色表示 實際人數 < 期望值人數, Observed < Expected. 例如: 頭等艙男性生還是紅色。
- # 灰色表示 實際人數 大約等於 期望值人數, Observed $\approx$ Expected, 即沒有顯著差異。
- # 圖例下方表示 $p\text{-value} < 2.22e-16$, 卡方獨立性檢定 (Chi-square Test of Independence) 的檢定結果。
- # 其意義為:
 - # 虛無假設 (H_0): Class、Sex、Age 與 Survived 彼此獨立, 沒有關聯。
 - # 對立假設 (H_1): Class、Sex、Age 與 Survived 彼此有關聯。
- # 檢定結果: $p\text{-value}$ 極小 (遠小於 0.05), 因此拒絕虛無假設。
- # 檢定結論: 艙等(Class), 性別(Sex), 年齡(Age) 與是否生還(Survived)之間存在顯著關聯。
- # 第1列結果:
 - # 1. 男性區塊比女性寬, 表示頭等艙男性乘客較多。
 - # 2. 女性的 Yes(生還) 區塊遠大於 No (死亡)。

3. 男性的 No (死亡) 區塊仍相當大。
 # 4. 幾乎全部都是 Adult (成人)，兒童非常少。
 # 5. Female No 深紅：死亡遠低於期望值。
 # 6. Female Yes 深藍：生還遠高於期望值。
 # 7. 第 1 列結論：男性沒有明顯偏向生還或死亡，女性呈現極端結果——幾乎都生還，很少死亡。

使用 ColorBrewer 配色
library(RColorBrewer)

```
mosaicplot(
  Titanic,
  # color = brewer.pal(8, "Set2"),
  color = brewer.pal(8, "Pastel1"),
  main = "Titanic Mosaic Plot with ColorBrewer"
)
```



將 table 轉換為 data.frame
 titanic_df <- **as.data.frame**(Titanic)
 titanic_df

##	Class	Sex	Age	Survived	Freq
## 1	1st	Male	Child	No	0
## 2	2nd	Male	Child	No	0
## 3	3rd	Male	Child	No	35
## 4	Crew	Male	Child	No	0

```
## 5    1st Female Child      No    0
## 6    2nd Female Child      No    0
## 7    3rd Female Child      No   17
## 8    Crew Female Child      No    0
## 9    1st  Male Adult       No  118
## 10   2nd  Male Adult       No  154
## 11   3rd  Male Adult       No  387
## 12   Crew  Male Adult       No  670
## 13   1st Female Adult      No    4
## 14   2nd Female Adult      No   13
## 15   3rd Female Adult      No   89
## 16   Crew Female Adult      No    3
## 17   1st  Male Child       Yes    5
## 18   2nd  Male Child       Yes   11
## 19   3rd  Male Child       Yes   13
## 20   Crew  Male Child       Yes    0
## 21   1st Female Child      Yes    1
## 22   2nd Female Child      Yes   13
## 23   3rd Female Child      Yes   14
## 24   Crew Female Child      Yes    0
## 25   1st  Male Adult       Yes   57
## 26   2nd  Male Adult       Yes   14
## 27   3rd  Male Adult       Yes   75
## 28   Crew  Male Adult       Yes  192
## 29   1st Female Adult      Yes  140
## 30   2nd Female Adult      Yes   80
## 31   3rd Female Adult      Yes   76
## 32   Crew Female Adult      Yes   20
```

3. airquality 1973 年紐約空氣品質資料集

```
?airquality # 153*6
```

```
data(airquality)
```

```
airquality # 153*6
```

```
##      Ozone Solar.R Wind Temp Month Day
## 1      41      190  7.4   67     5   1
## 2      36      118  8.0   72     5   2
## 3      12      149 12.6   74     5   3
## 4      18      313 11.5   62     5   4
## 5      NA       NA 14.3   56     5   5
## 6      28       NA 14.9   66     5   6
## 7      23      299  8.6   65     5   7
## 8      19       99 13.8   59     5   8
## 9       8       19 20.1   61     5   9
## 10     NA      194  8.6   69     5  10
## 11      7       NA  6.9   74     5  11
## 12     16      256  9.7   69     5  12
```


## 13	11	290	9.2	66	5	13
## 14	14	274	10.9	68	5	14
## 15	18	65	13.2	58	5	15
## 16	14	334	11.5	64	5	16
## 17	34	307	12.0	66	5	17
## 18	6	78	18.4	57	5	18
## 19	30	322	11.5	68	5	19
## 20	11	44	9.7	62	5	20
## 21	1	8	9.7	59	5	21
## 22	11	320	16.6	73	5	22
## 23	4	25	9.7	61	5	23
## 24	32	92	12.0	61	5	24
## 25	NA	66	16.6	57	5	25
## 26	NA	266	14.9	58	5	26
## 27	NA	NA	8.0	57	5	27
## 28	23	13	12.0	67	5	28
## 29	45	252	14.9	81	5	29
## 30	115	223	5.7	79	5	30
## 31	37	279	7.4	76	5	31
## 32	NA	286	8.6	78	6	1
## 33	NA	287	9.7	74	6	2
## 34	NA	242	16.1	67	6	3
## 35	NA	186	9.2	84	6	4
## 36	NA	220	8.6	85	6	5
## 37	NA	264	14.3	79	6	6
## 38	29	127	9.7	82	6	7
## 39	NA	273	6.9	87	6	8
## 40	71	291	13.8	90	6	9
## 41	39	323	11.5	87	6	10
## 42	NA	259	10.9	93	6	11
## 43	NA	250	9.2	92	6	12
## 44	23	148	8.0	82	6	13
## 45	NA	332	13.8	80	6	14
## 46	NA	322	11.5	79	6	15
## 47	21	191	14.9	77	6	16
## 48	37	284	20.7	72	6	17
## 49	20	37	9.2	65	6	18
## 50	12	120	11.5	73	6	19
## 51	13	137	10.3	76	6	20
## 52	NA	150	6.3	77	6	21
## 53	NA	59	1.7	76	6	22
## 54	NA	91	4.6	76	6	23
## 55	NA	250	6.3	76	6	24
## 56	NA	135	8.0	75	6	25
## 57	NA	127	8.0	78	6	26
## 58	NA	47	10.3	73	6	27
## 59	NA	98	11.5	80	6	28
## 60	NA	31	14.9	77	6	29
## 61	NA	138	8.0	83	6	30
## 62	135	269	4.1	84	7	1

## 63	49	248	9.2	85	7	2
## 64	32	236	9.2	81	7	3
## 65	NA	101	10.9	84	7	4
## 66	64	175	4.6	83	7	5
## 67	40	314	10.9	83	7	6
## 68	77	276	5.1	88	7	7
## 69	97	267	6.3	92	7	8
## 70	97	272	5.7	92	7	9
## 71	85	175	7.4	89	7	10
## 72	NA	139	8.6	82	7	11
## 73	10	264	14.3	73	7	12
## 74	27	175	14.9	81	7	13
## 75	NA	291	14.9	91	7	14
## 76	7	48	14.3	80	7	15
## 77	48	260	6.9	81	7	16
## 78	35	274	10.3	82	7	17
## 79	61	285	6.3	84	7	18
## 80	79	187	5.1	87	7	19
## 81	63	220	11.5	85	7	20
## 82	16	7	6.9	74	7	21
## 83	NA	258	9.7	81	7	22
## 84	NA	295	11.5	82	7	23
## 85	80	294	8.6	86	7	24
## 86	108	223	8.0	85	7	25
## 87	20	81	8.6	82	7	26
## 88	52	82	12.0	86	7	27
## 89	82	213	7.4	88	7	28
## 90	50	275	7.4	86	7	29
## 91	64	253	7.4	83	7	30
## 92	59	254	9.2	81	7	31
## 93	39	83	6.9	81	8	1
## 94	9	24	13.8	81	8	2
## 95	16	77	7.4	82	8	3
## 96	78	NA	6.9	86	8	4
## 97	35	NA	7.4	85	8	5
## 98	66	NA	4.6	87	8	6
## 99	122	255	4.0	89	8	7
## 100	89	229	10.3	90	8	8
## 101	110	207	8.0	90	8	9
## 102	NA	222	8.6	92	8	10
## 103	NA	137	11.5	86	8	11
## 104	44	192	11.5	86	8	12
## 105	28	273	11.5	82	8	13
## 106	65	157	9.7	80	8	14
## 107	NA	64	11.5	79	8	15
## 108	22	71	10.3	77	8	16
## 109	59	51	6.3	79	8	17
## 110	23	115	7.4	76	8	18
## 111	31	244	10.9	78	8	19
## 112	44	190	10.3	78	8	20

```

## 113    21    259 15.5    77     8    21
## 114     9     36 14.3    72     8    22
## 115    NA    255 12.6    75     8    23
## 116    45    212  9.7    79     8    24
## 117   168    238  3.4    81     8    25
## 118    73    215  8.0    86     8    26
## 119    NA    153  5.7    88     8    27
## 120    76    203  9.7    97     8    28
## 121   118    225  2.3    94     8    29
## 122    84    237  6.3    96     8    30
## 123    85    188  6.3    94     8    31
## 124    96    167  6.9    91     9     1
## 125    78    197  5.1    92     9     2
## 126    73    183  2.8    93     9     3
## 127    91    189  4.6    93     9     4
## 128    47     95  7.4    87     9     5
## 129    32     92 15.5    84     9     6
## 130    20    252 10.9    80     9     7
## 131    23    220 10.3    78     9     8
## 132    21    230 10.9    75     9     9
## 133    24    259  9.7    73     9    10
## 134    44    236 14.9    81     9    11
## 135    21    259 15.5    76     9    12
## 136    28    238  6.3    77     9    13
## 137     9     24 10.9    71     9    14
## 138    13    112 11.5    71     9    15
## 139    46    237  6.9    78     9    16
## 140    18    224 13.8    67     9    17
## 141    13     27 10.3    76     9    18
## 142    24    238 10.3    68     9    19
## 143    16    201  8.0    82     9    20
## 144    13    238 12.6    64     9    21
## 145    23     14  9.2    71     9    22
## 146    36    139 10.3    81     9    23
## 147     7     49 10.3    69     9    24
## 148    14     20 16.6    63     9    25
## 149    30    193  6.9    70     9    26
## 150    NA    145 13.2    77     9    27
## 151    14    191 14.3    75     9    28
## 152    18    131  8.0    76     9    29
## 153    20    223 11.5    68     9    30

class(airquality)

## [1] "data.frame"

str(airquality)

## 'data.frame':   153 obs. of  6 variables:
##  $ Ozone   : int  41 36 12 18 NA 28 23 19 8 NA ...
##  $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...

```

```
## $ Wind : num 7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
## $ Temp : int 67 72 74 62 56 66 65 59 61 69 ...
## $ Month : int 5 5 5 5 5 5 5 5 5 5 ...
## $ Day : int 1 2 3 4 5 6 7 8 9 10 ...
```

```
# Ozone : Ozone (ppb) : 平均臭氧濃度
# Solar.R : Solar R (lang) : 太陽輻射頻率
# Wind : Wind (mph) : 平均風速(英里/小時)
# Temp : Temperature (degrees F) : 每日最高溫(華氏)
# Month : Month (1--12) : 月份
# Day : Day of month (1--31) : 日期
```

```
# 有 NA 值
```

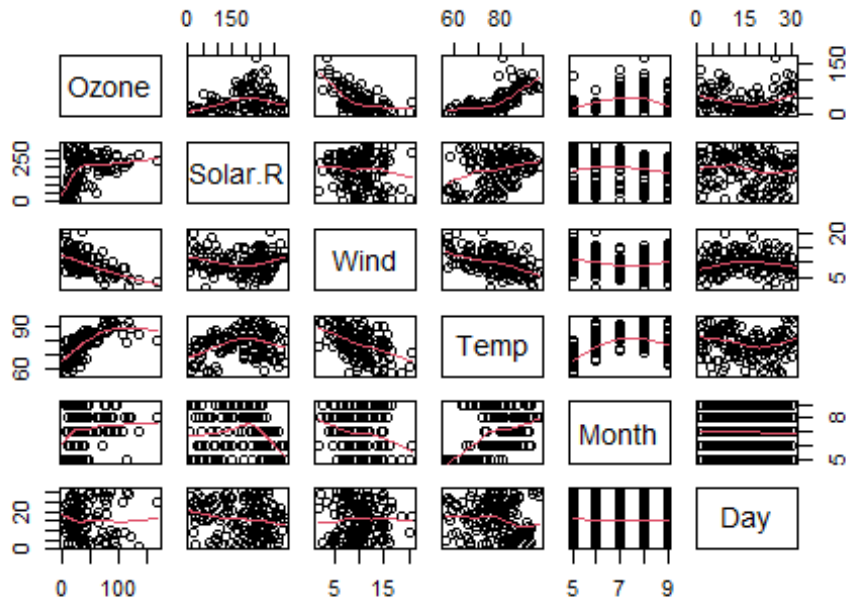
```
summary(airquality)
```

```
##      Ozone      Solar.R      Wind      Temp
## Min.   : 1.00    Min.   : 7.0    Min.   : 1.700   Min.   :56.00
## 1st Qu.: 18.00    1st Qu.:115.8    1st Qu.: 7.400   1st Qu.:72.00
## Median : 31.50    Median :205.0    Median : 9.700   Median :79.00
## Mean   : 42.13    Mean   :185.9    Mean   : 9.958   Mean   :77.88
## 3rd Qu.: 63.25    3rd Qu.:258.8    3rd Qu.:11.500   3rd Qu.:85.00
## Max.   :168.00    Max.   :334.0    Max.   :20.700   Max.   :97.00
## NAs    :37       NAs    :7
##      Month      Day
## Min.   :5.000    Min.   : 1.0
## 1st Qu.:6.000    1st Qu.: 8.0
## Median :7.000    Median :16.0
## Mean   :6.993    Mean   :15.8
## 3rd Qu.:8.000    3rd Qu.:23.0
## Max.   :9.000    Max.   :31.0
##
```

```
# 散佈圖矩陣 (Scatterplot Matrices)
```

```
pairs(airquality,
      panel = panel.smooth,
      main = "1973 年紐約空氣品質-散佈圖矩陣")
```

1973年紐約空氣品質-散佈圖矩陣



找出樣式 (Patterns)

4. iris 鳶尾花資料集——

```
?iris
```

```
data(iris)
```

```
iris # 150*5
```

##	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
## 1	5.1	3.5	1.4	0.2	setosa
## 2	4.9	3.0	1.4	0.2	setosa
## 3	4.7	3.2	1.3	0.2	setosa
## 4	4.6	3.1	1.5	0.2	setosa
## 5	5.0	3.6	1.4	0.2	setosa
## 6	5.4	3.9	1.7	0.4	setosa
## 7	4.6	3.4	1.4	0.3	setosa
## 8	5.0	3.4	1.5	0.2	setosa
## 9	4.4	2.9	1.4	0.2	setosa
## 10	4.9	3.1	1.5	0.1	setosa
## 11	5.4	3.7	1.5	0.2	setosa
## 12	4.8	3.4	1.6	0.2	setosa
## 13	4.8	3.0	1.4	0.1	setosa
## 14	4.3	3.0	1.1	0.1	setosa
## 15	5.8	4.0	1.2	0.2	setosa
## 16	5.7	4.4	1.5	0.4	setosa

## 17	5.4	3.9	1.3	0.4	setosa
## 18	5.1	3.5	1.4	0.3	setosa
## 19	5.7	3.8	1.7	0.3	setosa
## 20	5.1	3.8	1.5	0.3	setosa
## 21	5.4	3.4	1.7	0.2	setosa
## 22	5.1	3.7	1.5	0.4	setosa
## 23	4.6	3.6	1.0	0.2	setosa
## 24	5.1	3.3	1.7	0.5	setosa
## 25	4.8	3.4	1.9	0.2	setosa
## 26	5.0	3.0	1.6	0.2	setosa
## 27	5.0	3.4	1.6	0.4	setosa
## 28	5.2	3.5	1.5	0.2	setosa
## 29	5.2	3.4	1.4	0.2	setosa
## 30	4.7	3.2	1.6	0.2	setosa
## 31	4.8	3.1	1.6	0.2	setosa
## 32	5.4	3.4	1.5	0.4	setosa
## 33	5.2	4.1	1.5	0.1	setosa
## 34	5.5	4.2	1.4	0.2	setosa
## 35	4.9	3.1	1.5	0.2	setosa
## 36	5.0	3.2	1.2	0.2	setosa
## 37	5.5	3.5	1.3	0.2	setosa
## 38	4.9	3.6	1.4	0.1	setosa
## 39	4.4	3.0	1.3	0.2	setosa
## 40	5.1	3.4	1.5	0.2	setosa
## 41	5.0	3.5	1.3	0.3	setosa
## 42	4.5	2.3	1.3	0.3	setosa
## 43	4.4	3.2	1.3	0.2	setosa
## 44	5.0	3.5	1.6	0.6	setosa
## 45	5.1	3.8	1.9	0.4	setosa
## 46	4.8	3.0	1.4	0.3	setosa
## 47	5.1	3.8	1.6	0.2	setosa
## 48	4.6	3.2	1.4	0.2	setosa
## 49	5.3	3.7	1.5	0.2	setosa
## 50	5.0	3.3	1.4	0.2	setosa
## 51	7.0	3.2	4.7	1.4	versicolor
## 52	6.4	3.2	4.5	1.5	versicolor
## 53	6.9	3.1	4.9	1.5	versicolor
## 54	5.5	2.3	4.0	1.3	versicolor
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 58	4.9	2.4	3.3	1.0	versicolor
## 59	6.6	2.9	4.6	1.3	versicolor
## 60	5.2	2.7	3.9	1.4	versicolor
## 61	5.0	2.0	3.5	1.0	versicolor
## 62	5.9	3.0	4.2	1.5	versicolor
## 63	6.0	2.2	4.0	1.0	versicolor
## 64	6.1	2.9	4.7	1.4	versicolor
## 65	5.6	2.9	3.6	1.3	versicolor
## 66	6.7	3.1	4.4	1.4	versicolor

## 67	5.6	3.0	4.5	1.5 versicolor
## 68	5.8	2.7	4.1	1.0 versicolor
## 69	6.2	2.2	4.5	1.5 versicolor
## 70	5.6	2.5	3.9	1.1 versicolor
## 71	5.9	3.2	4.8	1.8 versicolor
## 72	6.1	2.8	4.0	1.3 versicolor
## 73	6.3	2.5	4.9	1.5 versicolor
## 74	6.1	2.8	4.7	1.2 versicolor
## 75	6.4	2.9	4.3	1.3 versicolor
## 76	6.6	3.0	4.4	1.4 versicolor
## 77	6.8	2.8	4.8	1.4 versicolor
## 78	6.7	3.0	5.0	1.7 versicolor
## 79	6.0	2.9	4.5	1.5 versicolor
## 80	5.7	2.6	3.5	1.0 versicolor
## 81	5.5	2.4	3.8	1.1 versicolor
## 82	5.5	2.4	3.7	1.0 versicolor
## 83	5.8	2.7	3.9	1.2 versicolor
## 84	6.0	2.7	5.1	1.6 versicolor
## 85	5.4	3.0	4.5	1.5 versicolor
## 86	6.0	3.4	4.5	1.6 versicolor
## 87	6.7	3.1	4.7	1.5 versicolor
## 88	6.3	2.3	4.4	1.3 versicolor
## 89	5.6	3.0	4.1	1.3 versicolor
## 90	5.5	2.5	4.0	1.3 versicolor
## 91	5.5	2.6	4.4	1.2 versicolor
## 92	6.1	3.0	4.6	1.4 versicolor
## 93	5.8	2.6	4.0	1.2 versicolor
## 94	5.0	2.3	3.3	1.0 versicolor
## 95	5.6	2.7	4.2	1.3 versicolor
## 96	5.7	3.0	4.2	1.2 versicolor
## 97	5.7	2.9	4.2	1.3 versicolor
## 98	6.2	2.9	4.3	1.3 versicolor
## 99	5.1	2.5	3.0	1.1 versicolor
## 100	5.7	2.8	4.1	1.3 versicolor
## 101	6.3	3.3	6.0	2.5 virginica
## 102	5.8	2.7	5.1	1.9 virginica
## 103	7.1	3.0	5.9	2.1 virginica
## 104	6.3	2.9	5.6	1.8 virginica
## 105	6.5	3.0	5.8	2.2 virginica
## 106	7.6	3.0	6.6	2.1 virginica
## 107	4.9	2.5	4.5	1.7 virginica
## 108	7.3	2.9	6.3	1.8 virginica
## 109	6.7	2.5	5.8	1.8 virginica
## 110	7.2	3.6	6.1	2.5 virginica
## 111	6.5	3.2	5.1	2.0 virginica
## 112	6.4	2.7	5.3	1.9 virginica
## 113	6.8	3.0	5.5	2.1 virginica
## 114	5.7	2.5	5.0	2.0 virginica
## 115	5.8	2.8	5.1	2.4 virginica
## 116	6.4	3.2	5.3	2.3 virginica

```
## 117      6.5      3.0      5.5      1.8 virginica
## 118      7.7      3.8      6.7      2.2 virginica
## 119      7.7      2.6      6.9      2.3 virginica
## 120      6.0      2.2      5.0      1.5 virginica
## 121      6.9      3.2      5.7      2.3 virginica
## 122      5.6      2.8      4.9      2.0 virginica
## 123      7.7      2.8      6.7      2.0 virginica
## 124      6.3      2.7      4.9      1.8 virginica
## 125      6.7      3.3      5.7      2.1 virginica
## 126      7.2      3.2      6.0      1.8 virginica
## 127      6.2      2.8      4.8      1.8 virginica
## 128      6.1      3.0      4.9      1.8 virginica
## 129      6.4      2.8      5.6      2.1 virginica
## 130      7.2      3.0      5.8      1.6 virginica
## 131      7.4      2.8      6.1      1.9 virginica
## 132      7.9      3.8      6.4      2.0 virginica
## 133      6.4      2.8      5.6      2.2 virginica
## 134      6.3      2.8      5.1      1.5 virginica
## 135      6.1      2.6      5.6      1.4 virginica
## 136      7.7      3.0      6.1      2.3 virginica
## 137      6.3      3.4      5.6      2.4 virginica
## 138      6.4      3.1      5.5      1.8 virginica
## 139      6.0      3.0      4.8      1.8 virginica
## 140      6.9      3.1      5.4      2.1 virginica
## 141      6.7      3.1      5.6      2.4 virginica
## 142      6.9      3.1      5.1      2.3 virginica
## 143      5.8      2.7      5.1      1.9 virginica
## 144      6.8      3.2      5.9      2.3 virginica
## 145      6.7      3.3      5.7      2.5 virginica
## 146      6.7      3.0      5.2      2.3 virginica
## 147      6.3      2.5      5.0      1.9 virginica
## 148      6.5      3.0      5.2      2.0 virginica
## 149      6.2      3.4      5.4      2.3 virginica
## 150      5.9      3.0      5.1      1.8 virginica
```

`str(iris)` # Species 為 factor 物件, 有3種花 ("setosa", "versicolor", "virginica")

```
## 'data.frame':   150 obs. of  5 variables:
## $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
## $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
## $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
## $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
## $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1
1 1 1 1 1 1 1 ...
```

`levels(iris$Species)`

```
## [1] "setosa"      "versicolor" "virginica"
```

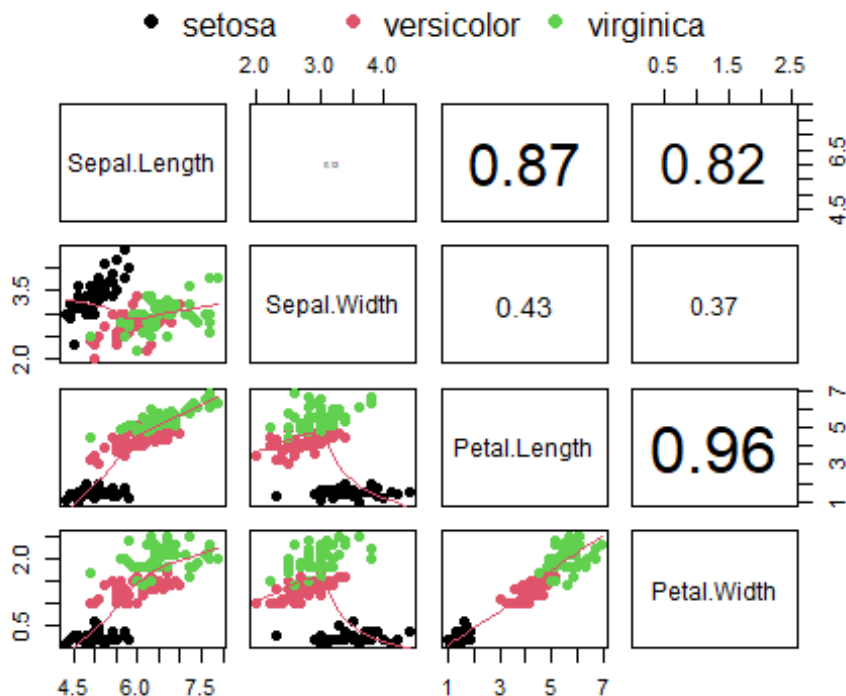
`summary(iris)` # 沒有 NA 值, 3種花的樣本數皆各為50筆.


```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
## Min. :4.300 Min. :2.000 Min. :1.000 Min. :0.100
## 1st Qu.:5.100 1st Qu.:2.800 1st Qu.:1.600 1st Qu.:0.300
## Median :5.800 Median :3.000 Median :4.350 Median :1.300
## Mean :5.843 Mean :3.057 Mean :3.758 Mean :1.199
## 3rd Qu.:6.400 3rd Qu.:3.300 3rd Qu.:5.100 3rd Qu.:1.800
## Max. :7.900 Max. :4.400 Max. :6.900 Max. :2.500
## Species
## setosa :50
## versicolor:50
## virginica :50
##
##
##

# 相關係數
panel.cor <- function(x, y, digits = 2, prefix = "", cex.cor, ...) {
  par(usr = c(0, 1, 0, 1))
  r <- abs(cor(x, y))
  txt <- format(c(r, 0.123456789), digits = digits)[1]
  txt <- paste0(prefix, txt)
  if(missing(cex.cor)) cex.cor <- 0.8/strwidth(txt)
  text(0.5, 0.5, txt, cex = cex.cor * r)
}

# iris-散佈圖矩陣
pairs(iris[1:4],
      col = iris$Species,
      pch = 16,
      lower.panel = panel.smooth, # 使用 Lowess
      upper.panel = panel.cor)

legend(x = 0.5,
       y = 1,
       levels(iris$Species),
       pch = 16,
       col = (1:3),
       bty = "n",
       horiz = TRUE,
       xjust = 0.5,
       yjust = 0.2,
       xpd = TRUE)
```



```
# bty = "n" 取消圖例外框
# xpd = TRUE: 所有繪圖都會被裁切至圖形區域(figure region)
# xpd = FALSE: 所有繪圖都會被裁切至繪圖(plot region)
# 找出樣式 (Patterns)
```

5. mtcars 汽車資料集

```
?mtcars
#
# mpg      : Miles/(US) gallon           ; 英里/加侖
# cyl      : Number of cylinders         ; 汽缸數
# disp     : Displacement (cu.in.)       ; 排氣量
# hp       : Gross horsepower            ; 總馬力
# drat     : Rear axle ratio              ; 後軸傳動比
# wt       : Weight (1000 lbs)            ; 重量
# qsec     : 1/4 mile time                ; 加速至 1/4 英哩時間
# vs       : Engine (0 = V-shaped, 1 = straight) ; 引擎排列
# am       : Transmission (0 = automatic, 1 = manual); 變速箱(0=自排, 1=手排)
# gear     : Number of forward gears      ; 前進檔數
# carb     : Number of carburetors        ; 化油器數量
data(mtcars)

mtcars # 32*11
```

## arb	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	c
## Mazda RX4 4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	
## Mazda RX4 Wag 4	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	
## Datsun 710 1	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	
## Hornet 4 Drive 1	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	
## Hornet Sportabout 2	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	
## Valiant 1	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	
## Duster 360 4	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	
## Merc 240D 2	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	
## Merc 230 2	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	
## Merc 280 4	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	
## Merc 280C 4	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	
## Merc 450SE 3	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	
## Merc 450SL 3	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	
## Merc 450SLC 3	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	
## Cadillac Fleetwood 4	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	
## Lincoln Continental 4	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	
## Chrysler Imperial 4	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	
## Fiat 128 1	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	
## Honda Civic 2	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	
## Toyota Corolla 1	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	
## Toyota Corona 1	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	
## Dodge Challenger 2	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	
## AMC Javelin 2	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	
## Camaro Z28 4	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	

```
## Pontiac Firebird      19.2   8 400.0 175 3.08 3.845 17.05  0  0   3
  2
## Fiat X1-9            27.3   4  79.0  66 4.08 1.935 18.90  1  1   4
  1
## Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1   5
  2
## Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1   5
  2
## Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1   5
  4
## Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1   5
  6
## Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1   5
  8
## Volvo 142E         21.4   4 121.0 109 4.11 2.780 18.60  1  1   4
  2
```

`str(mtcars)` # 所有資料皆為數值型

```
## 'data.frame':   32 obs. of  11 variables:
## $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
## $ cyl : num   6  6  4  6  8  6  8  4  4  6 ...
## $ disp: num  160 160 108 258 360 ...
## $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
## $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
## $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
## $ qsec: num  16.5 17 18.6 19.4 17 ...
## $ vs  : num   0  0  1  1  0  1  0  1  1  1 ...
## $ am  : num   1  1  1  0  0  0  0  0  0  0 ...
## $ gear: num   4  4  4  3  3  3  3  4  4  4 ...
## $ carb: num   4  4  1  1  2  1  4  2  2  4 ...
```

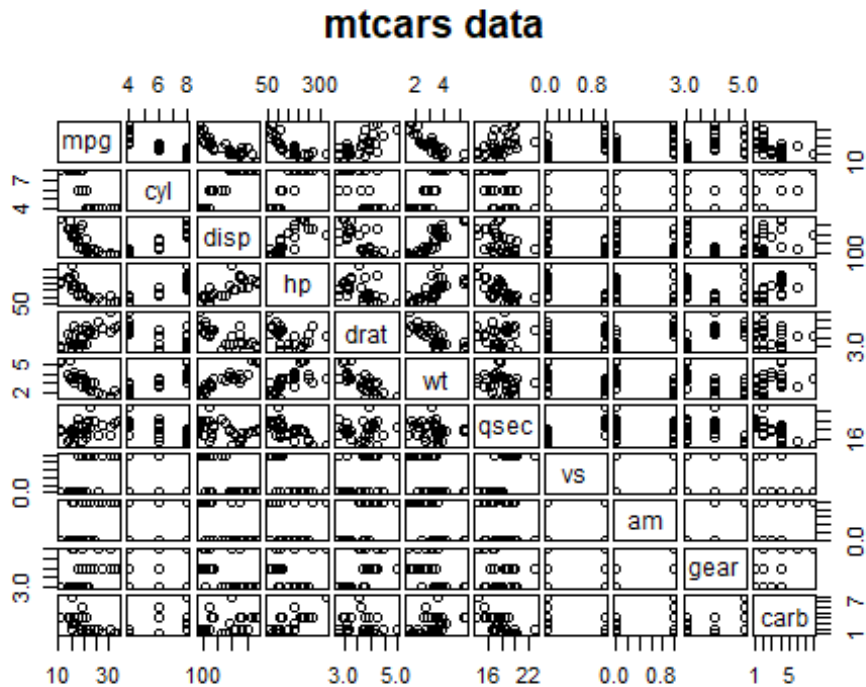
`summary(mtcars)` # 資料沒有 NA

```
##      mpg           cyl          disp          hp
## Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0
## 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
## Median :19.20   Median :6.000   Median :196.3   Median :123.0
## Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
## 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
## Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0
##      drat          wt          qsec          vs
## Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000
## 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
## Median :3.695   Median :3.325   Median :17.71   Median :0.0000
## Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375
## 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
## Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000
##      am          gear          carb
## Min.   :0.0000   Min.   :3.000   Min.   :1.000
```

```
## 1st Qu.:0.0000 1st Qu.:3.000 1st Qu.:2.000
## Median :0.0000 Median :4.000 Median :2.000
## Mean :0.4062 Mean :3.688 Mean :2.812
## 3rd Qu.:1.0000 3rd Qu.:4.000 3rd Qu.:4.000
## Max. :1.0000 Max. :5.000 Max. :8.000
```

mtcars-散佈圖矩陣

```
pairs(mtcars,
      main = "mtcars data",
      gap = 1/4)
```

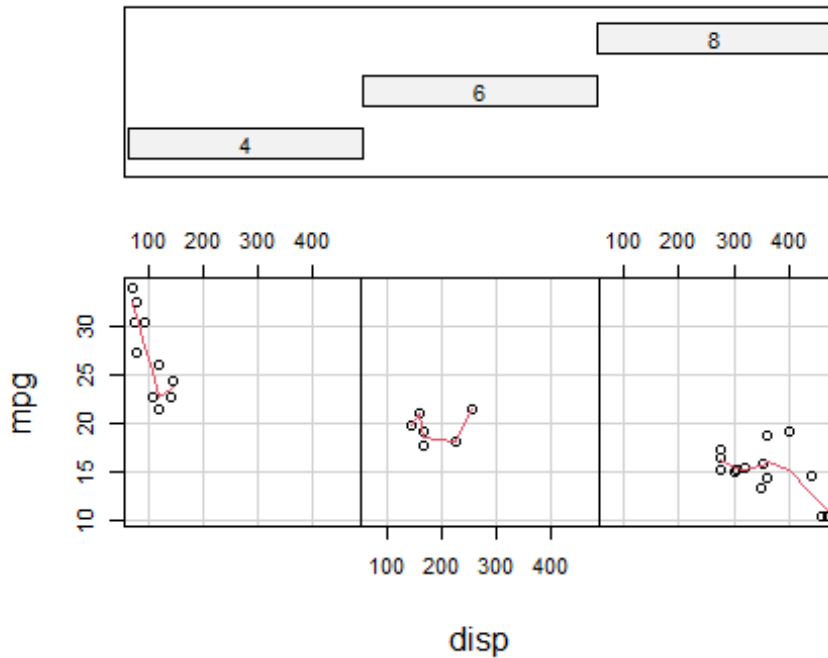


gap: 對角線上的變數名稱(Label)與左右散佈圖保留多少空白距離, 本例將預設留白縮小為 0.25, 整張圖會更緊湊。

mtcars-條件圖(區分不同 cyl 條件)

```
coplot(mpg ~ disp | as.factor(cyl),
      data = mtcars,
      panel = panel.smooth,
      rows = 1)
```

Given : `as.factor(cyl)`



6. penguins 南極洲帕爾默站企鵝

?penguins

```
# species: factor, with levels Adelie, Chinstrap, and Gentoo; 3 個物種:
# 阿德利企鵝(Adelie penguin), 頰帶企鵝(chinstrap penguin), 巴布亞企鵝(Gento
# o penguin)
# island: factor, with levels Biscoe, Dream, and Torgersen) ; 島嶼
# bill_len: numeric, bill length (millimeters) ; 喙長(公釐,
# mm)
# bill_dep: numeric, bill depth (millimeters) ; 喙深
# flipper_len: integer, flipper length (millimeters) ; 鰭長
# body_mass: integer, body mass (grams) ; 重量
# sex: factor, with levels female and male ; 性別
# year: integer, study year: 2007, 2008, or 2009 ; 研究年份
```

```
data(penguins)
```

penguins # 344*8

##	species	island	bill_len	bill_dep	flipper_len	body_mass	s
ex year							
## 1	Adelie	Torgersen	39.1	18.7	181	3750	ma
le 2007							
## 2	Adelie	Torgersen	39.5	17.4	186	3800	fema
le 2007							

## 3 le 2007	Adelie Torgersen	40.3	18.0	195	3250	fema
## 4 A> 2007	Adelie Torgersen	NA	NA	NA	NA	<N
## 5 le 2007	Adelie Torgersen	36.7	19.3	193	3450	fema
## 6 le 2007	Adelie Torgersen	39.3	20.6	190	3650	ma
## 7 le 2007	Adelie Torgersen	38.9	17.8	181	3625	fema
## 8 le 2007	Adelie Torgersen	39.2	19.6	195	4675	ma
## 9 A> 2007	Adelie Torgersen	34.1	18.1	193	3475	<N
## 10 A> 2007	Adelie Torgersen	42.0	20.2	190	4250	<N
## 11 A> 2007	Adelie Torgersen	37.8	17.1	186	3300	<N
## 12 A> 2007	Adelie Torgersen	37.8	17.3	180	3700	<N
## 13 le 2007	Adelie Torgersen	41.1	17.6	182	3200	fema
## 14 le 2007	Adelie Torgersen	38.6	21.2	191	3800	ma
## 15 le 2007	Adelie Torgersen	34.6	21.1	198	4400	ma
## 16 le 2007	Adelie Torgersen	36.6	17.8	185	3700	fema
## 17 le 2007	Adelie Torgersen	38.7	19.0	195	3450	fema
## 18 le 2007	Adelie Torgersen	42.5	20.7	197	4500	ma
## 19 le 2007	Adelie Torgersen	34.4	18.4	184	3325	fema
## 20 le 2007	Adelie Torgersen	46.0	21.5	194	4200	ma
## 21 le 2007	Adelie Biscoe	37.8	18.3	174	3400	fema
## 22 le 2007	Adelie Biscoe	37.7	18.7	180	3600	ma
## 23 le 2007	Adelie Biscoe	35.9	19.2	189	3800	fema
## 24 le 2007	Adelie Biscoe	38.2	18.1	185	3950	ma
## 25 le 2007	Adelie Biscoe	38.8	17.2	180	3800	ma
## 26 le 2007	Adelie Biscoe	35.3	18.9	187	3800	fema
## 27 le 2007	Adelie Biscoe	40.6	18.6	183	3550	ma

## 28	Adelie	Biscoe	40.5	17.9	187	3200	fema
le 2007							
## 29	Adelie	Biscoe	37.9	18.6	172	3150	fema
le 2007							
## 30	Adelie	Biscoe	40.5	18.9	180	3950	ma
le 2007							
## 31	Adelie	Dream	39.5	16.7	178	3250	fema
le 2007							
## 32	Adelie	Dream	37.2	18.1	178	3900	ma
le 2007							
## 33	Adelie	Dream	39.5	17.8	188	3300	fema
le 2007							
## 34	Adelie	Dream	40.9	18.9	184	3900	ma
le 2007							
## 35	Adelie	Dream	36.4	17.0	195	3325	fema
le 2007							
## 36	Adelie	Dream	39.2	21.1	196	4150	ma
le 2007							
## 37	Adelie	Dream	38.8	20.0	190	3950	ma
le 2007							
## 38	Adelie	Dream	42.2	18.5	180	3550	fema
le 2007							
## 39	Adelie	Dream	37.6	19.3	181	3300	fema
le 2007							
## 40	Adelie	Dream	39.8	19.1	184	4650	ma
le 2007							
## 41	Adelie	Dream	36.5	18.0	182	3150	fema
le 2007							
## 42	Adelie	Dream	40.8	18.4	195	3900	ma
le 2007							
## 43	Adelie	Dream	36.0	18.5	186	3100	fema
le 2007							
## 44	Adelie	Dream	44.1	19.7	196	4400	ma
le 2007							
## 45	Adelie	Dream	37.0	16.9	185	3000	fema
le 2007							
## 46	Adelie	Dream	39.6	18.8	190	4600	ma
le 2007							
## 47	Adelie	Dream	41.1	19.0	182	3425	ma
le 2007							
## 48	Adelie	Dream	37.5	18.9	179	2975	<N
A> 2007							
## 49	Adelie	Dream	36.0	17.9	190	3450	fema
le 2007							
## 50	Adelie	Dream	42.3	21.2	191	4150	ma
le 2007							
## 51	Adelie	Biscoe	39.6	17.7	186	3500	fema
le 2008							
## 52	Adelie	Biscoe	40.1	18.9	188	4300	ma
le 2008							

## 53 le 2008	Adelie	Biscoe	35.0	17.9	190	3450 fema
## 54 le 2008	Adelie	Biscoe	42.0	19.5	200	4050 ma
## 55 le 2008	Adelie	Biscoe	34.5	18.1	187	2900 fema
## 56 le 2008	Adelie	Biscoe	41.4	18.6	191	3700 ma
## 57 le 2008	Adelie	Biscoe	39.0	17.5	186	3550 fema
## 58 le 2008	Adelie	Biscoe	40.6	18.8	193	3800 ma
## 59 le 2008	Adelie	Biscoe	36.5	16.6	181	2850 fema
## 60 le 2008	Adelie	Biscoe	37.6	19.1	194	3750 ma
## 61 le 2008	Adelie	Biscoe	35.7	16.9	185	3150 fema
## 62 le 2008	Adelie	Biscoe	41.3	21.1	195	4400 ma
## 63 le 2008	Adelie	Biscoe	37.6	17.0	185	3600 fema
## 64 le 2008	Adelie	Biscoe	41.1	18.2	192	4050 ma
## 65 le 2008	Adelie	Biscoe	36.4	17.1	184	2850 fema
## 66 le 2008	Adelie	Biscoe	41.6	18.0	192	3950 ma
## 67 le 2008	Adelie	Biscoe	35.5	16.2	195	3350 fema
## 68 le 2008	Adelie	Biscoe	41.1	19.1	188	4100 ma
## 69 le 2008	Adelie	Torgersen	35.9	16.6	190	3050 fema
## 70 le 2008	Adelie	Torgersen	41.8	19.4	198	4450 ma
## 71 le 2008	Adelie	Torgersen	33.5	19.0	190	3600 fema
## 72 le 2008	Adelie	Torgersen	39.7	18.4	190	3900 ma
## 73 le 2008	Adelie	Torgersen	39.6	17.2	196	3550 fema
## 74 le 2008	Adelie	Torgersen	45.8	18.9	197	4150 ma
## 75 le 2008	Adelie	Torgersen	35.5	17.5	190	3700 fema
## 76 le 2008	Adelie	Torgersen	42.8	18.5	195	4250 ma
## 77 le 2008	Adelie	Torgersen	40.9	16.8	191	3700 fema

## 78	Adelie Torgersen	37.2	19.4	184	3900	ma
le 2008						
## 79	Adelie Torgersen	36.2	16.1	187	3550	fema
le 2008						
## 80	Adelie Torgersen	42.1	19.1	195	4000	ma
le 2008						
## 81	Adelie Torgersen	34.6	17.2	189	3200	fema
le 2008						
## 82	Adelie Torgersen	42.9	17.6	196	4700	ma
le 2008						
## 83	Adelie Torgersen	36.7	18.8	187	3800	fema
le 2008						
## 84	Adelie Torgersen	35.1	19.4	193	4200	ma
le 2008						
## 85	Adelie Dream	37.3	17.8	191	3350	fema
le 2008						
## 86	Adelie Dream	41.3	20.3	194	3550	ma
le 2008						
## 87	Adelie Dream	36.3	19.5	190	3800	ma
le 2008						
## 88	Adelie Dream	36.9	18.6	189	3500	fema
le 2008						
## 89	Adelie Dream	38.3	19.2	189	3950	ma
le 2008						
## 90	Adelie Dream	38.9	18.8	190	3600	fema
le 2008						
## 91	Adelie Dream	35.7	18.0	202	3550	fema
le 2008						
## 92	Adelie Dream	41.1	18.1	205	4300	ma
le 2008						
## 93	Adelie Dream	34.0	17.1	185	3400	fema
le 2008						
## 94	Adelie Dream	39.6	18.1	186	4450	ma
le 2008						
## 95	Adelie Dream	36.2	17.3	187	3300	fema
le 2008						
## 96	Adelie Dream	40.8	18.9	208	4300	ma
le 2008						
## 97	Adelie Dream	38.1	18.6	190	3700	fema
le 2008						
## 98	Adelie Dream	40.3	18.5	196	4350	ma
le 2008						
## 99	Adelie Dream	33.1	16.1	178	2900	fema
le 2008						
## 100	Adelie Dream	43.2	18.5	192	4100	ma
le 2008						
## 101	Adelie Biscoe	35.0	17.9	192	3725	fema
le 2009						
## 102	Adelie Biscoe	41.0	20.0	203	4725	ma
le 2009						

## 103	Adelie	Biscoe	37.7	16.0	183	3075	fema
le 2009							
## 104	Adelie	Biscoe	37.8	20.0	190	4250	ma
le 2009							
## 105	Adelie	Biscoe	37.9	18.6	193	2925	fema
le 2009							
## 106	Adelie	Biscoe	39.7	18.9	184	3550	ma
le 2009							
## 107	Adelie	Biscoe	38.6	17.2	199	3750	fema
le 2009							
## 108	Adelie	Biscoe	38.2	20.0	190	3900	ma
le 2009							
## 109	Adelie	Biscoe	38.1	17.0	181	3175	fema
le 2009							
## 110	Adelie	Biscoe	43.2	19.0	197	4775	ma
le 2009							
## 111	Adelie	Biscoe	38.1	16.5	198	3825	fema
le 2009							
## 112	Adelie	Biscoe	45.6	20.3	191	4600	ma
le 2009							
## 113	Adelie	Biscoe	39.7	17.7	193	3200	fema
le 2009							
## 114	Adelie	Biscoe	42.2	19.5	197	4275	ma
le 2009							
## 115	Adelie	Biscoe	39.6	20.7	191	3900	fema
le 2009							
## 116	Adelie	Biscoe	42.7	18.3	196	4075	ma
le 2009							
## 117	Adelie Torgersen		38.6	17.0	188	2900	fema
le 2009							
## 118	Adelie Torgersen		37.3	20.5	199	3775	ma
le 2009							
## 119	Adelie Torgersen		35.7	17.0	189	3350	fema
le 2009							
## 120	Adelie Torgersen		41.1	18.6	189	3325	ma
le 2009							
## 121	Adelie Torgersen		36.2	17.2	187	3150	fema
le 2009							
## 122	Adelie Torgersen		37.7	19.8	198	3500	ma
le 2009							
## 123	Adelie Torgersen		40.2	17.0	176	3450	fema
le 2009							
## 124	Adelie Torgersen		41.4	18.5	202	3875	ma
le 2009							
## 125	Adelie Torgersen		35.2	15.9	186	3050	fema
le 2009							
## 126	Adelie Torgersen		40.6	19.0	199	4000	ma
le 2009							
## 127	Adelie Torgersen		38.8	17.6	191	3275	fema
le 2009							

## 128	Adelie Torgersen	41.5	18.3	195	4300	ma
le 2009						
## 129	Adelie Torgersen	39.0	17.1	191	3050	fema
le 2009						
## 130	Adelie Torgersen	44.1	18.0	210	4000	ma
le 2009						
## 131	Adelie Torgersen	38.5	17.9	190	3325	fema
le 2009						
## 132	Adelie Torgersen	43.1	19.2	197	3500	ma
le 2009						
## 133	Adelie Dream	36.8	18.5	193	3500	fema
le 2009						
## 134	Adelie Dream	37.5	18.5	199	4475	ma
le 2009						
## 135	Adelie Dream	38.1	17.6	187	3425	fema
le 2009						
## 136	Adelie Dream	41.1	17.5	190	3900	ma
le 2009						
## 137	Adelie Dream	35.6	17.5	191	3175	fema
le 2009						
## 138	Adelie Dream	40.2	20.1	200	3975	ma
le 2009						
## 139	Adelie Dream	37.0	16.5	185	3400	fema
le 2009						
## 140	Adelie Dream	39.7	17.9	193	4250	ma
le 2009						
## 141	Adelie Dream	40.2	17.1	193	3400	fema
le 2009						
## 142	Adelie Dream	40.6	17.2	187	3475	ma
le 2009						
## 143	Adelie Dream	32.1	15.5	188	3050	fema
le 2009						
## 144	Adelie Dream	40.7	17.0	190	3725	ma
le 2009						
## 145	Adelie Dream	37.3	16.8	192	3000	fema
le 2009						
## 146	Adelie Dream	39.0	18.7	185	3650	ma
le 2009						
## 147	Adelie Dream	39.2	18.6	190	4250	ma
le 2009						
## 148	Adelie Dream	36.6	18.4	184	3475	fema
le 2009						
## 149	Adelie Dream	36.0	17.8	195	3450	fema
le 2009						
## 150	Adelie Dream	37.8	18.1	193	3750	ma
le 2009						
## 151	Adelie Dream	36.0	17.1	187	3700	fema
le 2009						
## 152	Adelie Dream	41.5	18.5	201	4000	ma
le 2009						

## 153	Gentoo	Biscoe	46.1	13.2	211	4500	fema
le 2007							
## 154	Gentoo	Biscoe	50.0	16.3	230	5700	ma
le 2007							
## 155	Gentoo	Biscoe	48.7	14.1	210	4450	fema
le 2007							
## 156	Gentoo	Biscoe	50.0	15.2	218	5700	ma
le 2007							
## 157	Gentoo	Biscoe	47.6	14.5	215	5400	ma
le 2007							
## 158	Gentoo	Biscoe	46.5	13.5	210	4550	fema
le 2007							
## 159	Gentoo	Biscoe	45.4	14.6	211	4800	fema
le 2007							
## 160	Gentoo	Biscoe	46.7	15.3	219	5200	ma
le 2007							
## 161	Gentoo	Biscoe	43.3	13.4	209	4400	fema
le 2007							
## 162	Gentoo	Biscoe	46.8	15.4	215	5150	ma
le 2007							
## 163	Gentoo	Biscoe	40.9	13.7	214	4650	fema
le 2007							
## 164	Gentoo	Biscoe	49.0	16.1	216	5550	ma
le 2007							
## 165	Gentoo	Biscoe	45.5	13.7	214	4650	fema
le 2007							
## 166	Gentoo	Biscoe	48.4	14.6	213	5850	ma
le 2007							
## 167	Gentoo	Biscoe	45.8	14.6	210	4200	fema
le 2007							
## 168	Gentoo	Biscoe	49.3	15.7	217	5850	ma
le 2007							
## 169	Gentoo	Biscoe	42.0	13.5	210	4150	fema
le 2007							
## 170	Gentoo	Biscoe	49.2	15.2	221	6300	ma
le 2007							
## 171	Gentoo	Biscoe	46.2	14.5	209	4800	fema
le 2007							
## 172	Gentoo	Biscoe	48.7	15.1	222	5350	ma
le 2007							
## 173	Gentoo	Biscoe	50.2	14.3	218	5700	ma
le 2007							
## 174	Gentoo	Biscoe	45.1	14.5	215	5000	fema
le 2007							
## 175	Gentoo	Biscoe	46.5	14.5	213	4400	fema
le 2007							
## 176	Gentoo	Biscoe	46.3	15.8	215	5050	ma
le 2007							
## 177	Gentoo	Biscoe	42.9	13.1	215	5000	fema
le 2007							

## 178	Gentoo	Biscoe	46.1	15.1	215	5100	ma
le 2007							
## 179	Gentoo	Biscoe	44.5	14.3	216	4100	<N
A> 2007							
## 180	Gentoo	Biscoe	47.8	15.0	215	5650	ma
le 2007							
## 181	Gentoo	Biscoe	48.2	14.3	210	4600	fema
le 2007							
## 182	Gentoo	Biscoe	50.0	15.3	220	5550	ma
le 2007							
## 183	Gentoo	Biscoe	47.3	15.3	222	5250	ma
le 2007							
## 184	Gentoo	Biscoe	42.8	14.2	209	4700	fema
le 2007							
## 185	Gentoo	Biscoe	45.1	14.5	207	5050	fema
le 2007							
## 186	Gentoo	Biscoe	59.6	17.0	230	6050	ma
le 2007							
## 187	Gentoo	Biscoe	49.1	14.8	220	5150	fema
le 2008							
## 188	Gentoo	Biscoe	48.4	16.3	220	5400	ma
le 2008							
## 189	Gentoo	Biscoe	42.6	13.7	213	4950	fema
le 2008							
## 190	Gentoo	Biscoe	44.4	17.3	219	5250	ma
le 2008							
## 191	Gentoo	Biscoe	44.0	13.6	208	4350	fema
le 2008							
## 192	Gentoo	Biscoe	48.7	15.7	208	5350	ma
le 2008							
## 193	Gentoo	Biscoe	42.7	13.7	208	3950	fema
le 2008							
## 194	Gentoo	Biscoe	49.6	16.0	225	5700	ma
le 2008							
## 195	Gentoo	Biscoe	45.3	13.7	210	4300	fema
le 2008							
## 196	Gentoo	Biscoe	49.6	15.0	216	4750	ma
le 2008							
## 197	Gentoo	Biscoe	50.5	15.9	222	5550	ma
le 2008							
## 198	Gentoo	Biscoe	43.6	13.9	217	4900	fema
le 2008							
## 199	Gentoo	Biscoe	45.5	13.9	210	4200	fema
le 2008							
## 200	Gentoo	Biscoe	50.5	15.9	225	5400	ma
le 2008							
## 201	Gentoo	Biscoe	44.9	13.3	213	5100	fema
le 2008							
## 202	Gentoo	Biscoe	45.2	15.8	215	5300	ma
le 2008							

## 203	Gentoo	Biscoe	46.6	14.2	210	4850	fema
le 2008							
## 204	Gentoo	Biscoe	48.5	14.1	220	5300	ma
le 2008							
## 205	Gentoo	Biscoe	45.1	14.4	210	4400	fema
le 2008							
## 206	Gentoo	Biscoe	50.1	15.0	225	5000	ma
le 2008							
## 207	Gentoo	Biscoe	46.5	14.4	217	4900	fema
le 2008							
## 208	Gentoo	Biscoe	45.0	15.4	220	5050	ma
le 2008							
## 209	Gentoo	Biscoe	43.8	13.9	208	4300	fema
le 2008							
## 210	Gentoo	Biscoe	45.5	15.0	220	5000	ma
le 2008							
## 211	Gentoo	Biscoe	43.2	14.5	208	4450	fema
le 2008							
## 212	Gentoo	Biscoe	50.4	15.3	224	5550	ma
le 2008							
## 213	Gentoo	Biscoe	45.3	13.8	208	4200	fema
le 2008							
## 214	Gentoo	Biscoe	46.2	14.9	221	5300	ma
le 2008							
## 215	Gentoo	Biscoe	45.7	13.9	214	4400	fema
le 2008							
## 216	Gentoo	Biscoe	54.3	15.7	231	5650	ma
le 2008							
## 217	Gentoo	Biscoe	45.8	14.2	219	4700	fema
le 2008							
## 218	Gentoo	Biscoe	49.8	16.8	230	5700	ma
le 2008							
## 219	Gentoo	Biscoe	46.2	14.4	214	4650	<N
A> 2008							
## 220	Gentoo	Biscoe	49.5	16.2	229	5800	ma
le 2008							
## 221	Gentoo	Biscoe	43.5	14.2	220	4700	fema
le 2008							
## 222	Gentoo	Biscoe	50.7	15.0	223	5550	ma
le 2008							
## 223	Gentoo	Biscoe	47.7	15.0	216	4750	fema
le 2008							
## 224	Gentoo	Biscoe	46.4	15.6	221	5000	ma
le 2008							
## 225	Gentoo	Biscoe	48.2	15.6	221	5100	ma
le 2008							
## 226	Gentoo	Biscoe	46.5	14.8	217	5200	fema
le 2008							
## 227	Gentoo	Biscoe	46.4	15.0	216	4700	fema
le 2008							

## 228	Gentoo	Biscoe	48.6	16.0	230	5800	ma
le 2008							
## 229	Gentoo	Biscoe	47.5	14.2	209	4600	fema
le 2008							
## 230	Gentoo	Biscoe	51.1	16.3	220	6000	ma
le 2008							
## 231	Gentoo	Biscoe	45.2	13.8	215	4750	fema
le 2008							
## 232	Gentoo	Biscoe	45.2	16.4	223	5950	ma
le 2008							
## 233	Gentoo	Biscoe	49.1	14.5	212	4625	fema
le 2009							
## 234	Gentoo	Biscoe	52.5	15.6	221	5450	ma
le 2009							
## 235	Gentoo	Biscoe	47.4	14.6	212	4725	fema
le 2009							
## 236	Gentoo	Biscoe	50.0	15.9	224	5350	ma
le 2009							
## 237	Gentoo	Biscoe	44.9	13.8	212	4750	fema
le 2009							
## 238	Gentoo	Biscoe	50.8	17.3	228	5600	ma
le 2009							
## 239	Gentoo	Biscoe	43.4	14.4	218	4600	fema
le 2009							
## 240	Gentoo	Biscoe	51.3	14.2	218	5300	ma
le 2009							
## 241	Gentoo	Biscoe	47.5	14.0	212	4875	fema
le 2009							
## 242	Gentoo	Biscoe	52.1	17.0	230	5550	ma
le 2009							
## 243	Gentoo	Biscoe	47.5	15.0	218	4950	fema
le 2009							
## 244	Gentoo	Biscoe	52.2	17.1	228	5400	ma
le 2009							
## 245	Gentoo	Biscoe	45.5	14.5	212	4750	fema
le 2009							
## 246	Gentoo	Biscoe	49.5	16.1	224	5650	ma
le 2009							
## 247	Gentoo	Biscoe	44.5	14.7	214	4850	fema
le 2009							
## 248	Gentoo	Biscoe	50.8	15.7	226	5200	ma
le 2009							
## 249	Gentoo	Biscoe	49.4	15.8	216	4925	ma
le 2009							
## 250	Gentoo	Biscoe	46.9	14.6	222	4875	fema
le 2009							
## 251	Gentoo	Biscoe	48.4	14.4	203	4625	fema
le 2009							
## 252	Gentoo	Biscoe	51.1	16.5	225	5250	ma
le 2009							

## 253	Gentoo	Biscoe	48.5	15.0	219	4850	fema
le 2009							
## 254	Gentoo	Biscoe	55.9	17.0	228	5600	ma
le 2009							
## 255	Gentoo	Biscoe	47.2	15.5	215	4975	fema
le 2009							
## 256	Gentoo	Biscoe	49.1	15.0	228	5500	ma
le 2009							
## 257	Gentoo	Biscoe	47.3	13.8	216	4725	<N
A> 2009							
## 258	Gentoo	Biscoe	46.8	16.1	215	5500	ma
le 2009							
## 259	Gentoo	Biscoe	41.7	14.7	210	4700	fema
le 2009							
## 260	Gentoo	Biscoe	53.4	15.8	219	5500	ma
le 2009							
## 261	Gentoo	Biscoe	43.3	14.0	208	4575	fema
le 2009							
## 262	Gentoo	Biscoe	48.1	15.1	209	5500	ma
le 2009							
## 263	Gentoo	Biscoe	50.5	15.2	216	5000	fema
le 2009							
## 264	Gentoo	Biscoe	49.8	15.9	229	5950	ma
le 2009							
## 265	Gentoo	Biscoe	43.5	15.2	213	4650	fema
le 2009							
## 266	Gentoo	Biscoe	51.5	16.3	230	5500	ma
le 2009							
## 267	Gentoo	Biscoe	46.2	14.1	217	4375	fema
le 2009							
## 268	Gentoo	Biscoe	55.1	16.0	230	5850	ma
le 2009							
## 269	Gentoo	Biscoe	44.5	15.7	217	4875	<N
A> 2009							
## 270	Gentoo	Biscoe	48.8	16.2	222	6000	ma
le 2009							
## 271	Gentoo	Biscoe	47.2	13.7	214	4925	fema
le 2009							
## 272	Gentoo	Biscoe	NA	NA	NA	NA	<N
A> 2009							
## 273	Gentoo	Biscoe	46.8	14.3	215	4850	fema
le 2009							
## 274	Gentoo	Biscoe	50.4	15.7	222	5750	ma
le 2009							
## 275	Gentoo	Biscoe	45.2	14.8	212	5200	fema
le 2009							
## 276	Gentoo	Biscoe	49.9	16.1	213	5400	ma
le 2009							
## 277	Chinstrap	Dream	46.5	17.9	192	3500	fema
le 2007							

## 278 Chinstrap le 2007	Dream	50.0	19.5	196	3900	ma
## 279 Chinstrap le 2007	Dream	51.3	19.2	193	3650	ma
## 280 Chinstrap le 2007	Dream	45.4	18.7	188	3525	fema
## 281 Chinstrap le 2007	Dream	52.7	19.8	197	3725	ma
## 282 Chinstrap le 2007	Dream	45.2	17.8	198	3950	fema
## 283 Chinstrap le 2007	Dream	46.1	18.2	178	3250	fema
## 284 Chinstrap le 2007	Dream	51.3	18.2	197	3750	ma
## 285 Chinstrap le 2007	Dream	46.0	18.9	195	4150	fema
## 286 Chinstrap le 2007	Dream	51.3	19.9	198	3700	ma
## 287 Chinstrap le 2007	Dream	46.6	17.8	193	3800	fema
## 288 Chinstrap le 2007	Dream	51.7	20.3	194	3775	ma
## 289 Chinstrap le 2007	Dream	47.0	17.3	185	3700	fema
## 290 Chinstrap le 2007	Dream	52.0	18.1	201	4050	ma
## 291 Chinstrap le 2007	Dream	45.9	17.1	190	3575	fema
## 292 Chinstrap le 2007	Dream	50.5	19.6	201	4050	ma
## 293 Chinstrap le 2007	Dream	50.3	20.0	197	3300	ma
## 294 Chinstrap le 2007	Dream	58.0	17.8	181	3700	fema
## 295 Chinstrap le 2007	Dream	46.4	18.6	190	3450	fema
## 296 Chinstrap le 2007	Dream	49.2	18.2	195	4400	ma
## 297 Chinstrap le 2007	Dream	42.4	17.3	181	3600	fema
## 298 Chinstrap le 2007	Dream	48.5	17.5	191	3400	ma
## 299 Chinstrap le 2007	Dream	43.2	16.6	187	2900	fema
## 300 Chinstrap le 2007	Dream	50.6	19.4	193	3800	ma
## 301 Chinstrap le 2007	Dream	46.7	17.9	195	3300	fema
## 302 Chinstrap le 2007	Dream	52.0	19.0	197	4150	ma

## 303 Chinstrap le 2008	Dream	50.5	18.4	200	3400 fema
## 304 Chinstrap le 2008	Dream	49.5	19.0	200	3800 ma
## 305 Chinstrap le 2008	Dream	46.4	17.8	191	3700 fema
## 306 Chinstrap le 2008	Dream	52.8	20.0	205	4550 ma
## 307 Chinstrap le 2008	Dream	40.9	16.6	187	3200 fema
## 308 Chinstrap le 2008	Dream	54.2	20.8	201	4300 ma
## 309 Chinstrap le 2008	Dream	42.5	16.7	187	3350 fema
## 310 Chinstrap le 2008	Dream	51.0	18.8	203	4100 ma
## 311 Chinstrap le 2008	Dream	49.7	18.6	195	3600 ma
## 312 Chinstrap le 2008	Dream	47.5	16.8	199	3900 fema
## 313 Chinstrap le 2008	Dream	47.6	18.3	195	3850 fema
## 314 Chinstrap le 2008	Dream	52.0	20.7	210	4800 ma
## 315 Chinstrap le 2008	Dream	46.9	16.6	192	2700 fema
## 316 Chinstrap le 2008	Dream	53.5	19.9	205	4500 ma
## 317 Chinstrap le 2008	Dream	49.0	19.5	210	3950 ma
## 318 Chinstrap le 2008	Dream	46.2	17.5	187	3650 fema
## 319 Chinstrap le 2008	Dream	50.9	19.1	196	3550 ma
## 320 Chinstrap le 2008	Dream	45.5	17.0	196	3500 fema
## 321 Chinstrap le 2009	Dream	50.9	17.9	196	3675 fema
## 322 Chinstrap le 2009	Dream	50.8	18.5	201	4450 ma
## 323 Chinstrap le 2009	Dream	50.1	17.9	190	3400 fema
## 324 Chinstrap le 2009	Dream	49.0	19.6	212	4300 ma
## 325 Chinstrap le 2009	Dream	51.5	18.7	187	3250 ma
## 326 Chinstrap le 2009	Dream	49.8	17.3	198	3675 fema
## 327 Chinstrap le 2009	Dream	48.1	16.4	199	3325 fema

```
## 328 Chinstrap Dream 51.4 19.0 201 3950 ma
le 2009
## 329 Chinstrap Dream 45.7 17.3 193 3600 fema
le 2009
## 330 Chinstrap Dream 50.7 19.7 203 4050 ma
le 2009
## 331 Chinstrap Dream 42.5 17.3 187 3350 fema
le 2009
## 332 Chinstrap Dream 52.2 18.8 197 3450 ma
le 2009
## 333 Chinstrap Dream 45.2 16.6 191 3250 fema
le 2009
## 334 Chinstrap Dream 49.3 19.9 203 4050 ma
le 2009
## 335 Chinstrap Dream 50.2 18.8 202 3800 ma
le 2009
## 336 Chinstrap Dream 45.6 19.4 194 3525 fema
le 2009
## 337 Chinstrap Dream 51.9 19.5 206 3950 ma
le 2009
## 338 Chinstrap Dream 46.8 16.5 189 3650 fema
le 2009
## 339 Chinstrap Dream 45.7 17.0 195 3650 fema
le 2009
## 340 Chinstrap Dream 55.8 19.8 207 4000 ma
le 2009
## 341 Chinstrap Dream 43.5 18.1 202 3400 fema
le 2009
## 342 Chinstrap Dream 49.6 18.2 193 3775 ma
le 2009
## 343 Chinstrap Dream 50.8 19.0 210 4100 ma
le 2009
## 344 Chinstrap Dream 50.2 18.7 198 3775 fema
le 2009
```

`str(penguins)` # 資料集包括 3 種不同資料型態: *Factor*, *num*, *int*

```
## 'data.frame': 344 obs. of 8 variables:
## $ species : Factor w/ 3 levels "Adelie","Chinstrap",...: 1 1 1 1
1 1 1 1 1 1 ...
## $ island : Factor w/ 3 levels "Biscoe","Dream",...: 3 3 3 3 3 3
3 3 3 3 ...
## $ bill_len : num 39.1 39.5 40.3 NA 36.7 39.3 38.9 39.2 34.1 42
...
## $ bill_dep : num 18.7 17.4 18 NA 19.3 20.6 17.8 19.6 18.1 20.2
...
## $ flipper_len: int 181 186 195 NA 193 190 181 195 193 190 ...
## $ body_mass : int 3750 3800 3250 NA 3450 3650 3625 4675 3475 4250
...
## $ sex : Factor w/ 2 levels "female","male": 2 1 1 NA 1 2 1 2
```

```
NA NA ...
## $ year      : int  2007 2007 2007 2007 2007 2007 2007 2007 2007 20
07 ...
```

```
summary(penguins) # 有 NA 值
```

```
##      species      island      bill_len      bill_dep
## Adelie   :152   Biscoe   :168   Min.    :32.10   Min.    :13.10
## Chinstrap: 68   Dream    :124   1st Qu.:39.23 1st Qu.:15.60
## Gentoo   :124   Torgersen: 52   Median :44.45 Median :17.30
##                                     Mean   :43.92 Mean   :17.15
##                                     3rd Qu.:48.50 3rd Qu.:18.70
##                                     Max.    :59.60 Max.    :21.50
##                                     NAs     :2     NAs     :2
## flipper_len  body_mass      sex      year
## Min.    :172.0   Min.    :2700   female:165   Min.    :2007
## 1st Qu.:190.0   1st Qu.:3550   male  :168   1st Qu.:2007
## Median :197.0   Median :4050   NAs    : 11   Median :2008
## Mean    :200.9   Mean    :4202                   Mean    :2008
## 3rd Qu.:213.0   3rd Qu.:4750                   3rd Qu.:2009
## Max.    :231.0   Max.    :6300                   Max.    :2009
## NAs      :2     NAs      :2
```

```
names(penguins)
```

```
## [1] "species"      "island"        "bill_len"      "bill_dep"      "flipper
_len"
## [6] "body_mass"    "sex"           "year"
```

```
# 各欄位之遺漏值數量
```

```
colSums(is.na(penguins))
```

```
##      species      island      bill_len      bill_dep flipper_len  body_m
ass
##           0           0           2           2           2
2
##           sex      year
##          11         0
```

```
# install.packages("GGally")
```

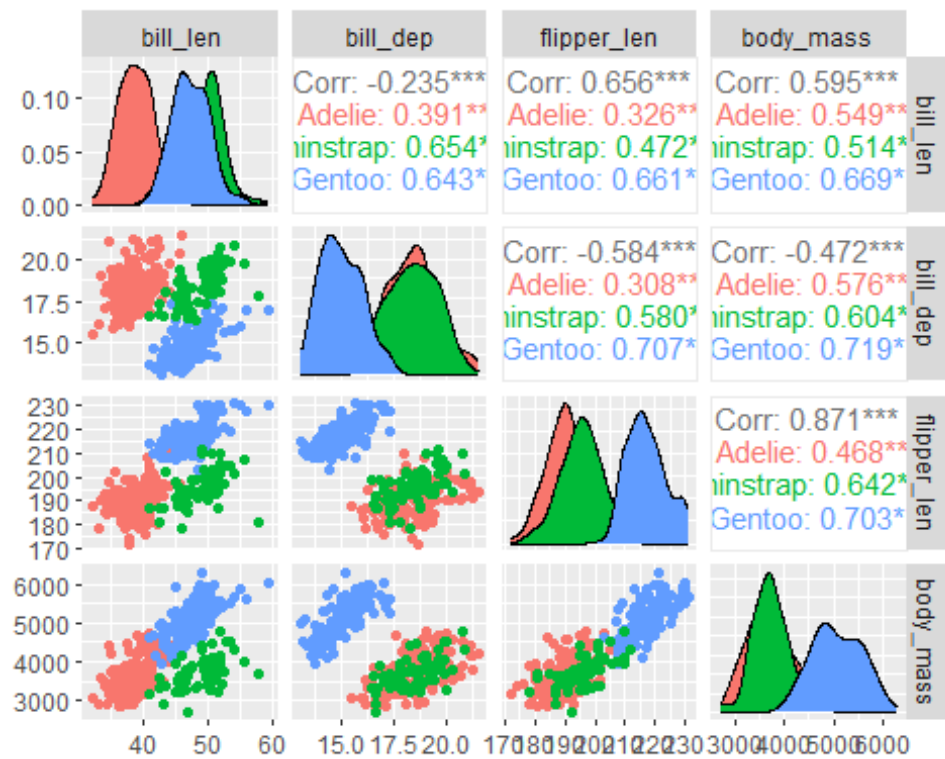
```
library(GGally)
```

```
ggpairs(
  penguins,
  columns =
    c("bill_len",
      "bill_dep",
      "flipper_len",
      "body_mass"),
```

```

    aes(color=species)
  )
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_density()`).
## Warning: Removed 2 rows containing missing values
## Removed 2 rows containing missing values
## Removed 2 rows containing missing values
## Warning: Removed 2 rows containing missing values or values outside
the scale range
## (`geom_point()`).
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_density()`).
## Warning: Removed 2 rows containing missing values
## Removed 2 rows containing missing values
## Warning: Removed 2 rows containing missing values or values outside
the scale range
## (`geom_point()`).
## Removed 2 rows containing missing values or values outside the scale
range
## (`geom_point()`).
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_density()`).
## Warning: Removed 2 rows containing missing values
## Warning: Removed 2 rows containing missing values or values outside
the scale range
## (`geom_point()`).
## Removed 2 rows containing missing values or values outside the scale
range
## (`geom_point()`).
## Removed 2 rows containing missing values or values outside the scale
range
## (`geom_point()`).
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_density()`).

```



```
# 找出樣式 (Patterns)
```

```
end
```